

Cache System

2013/2/18

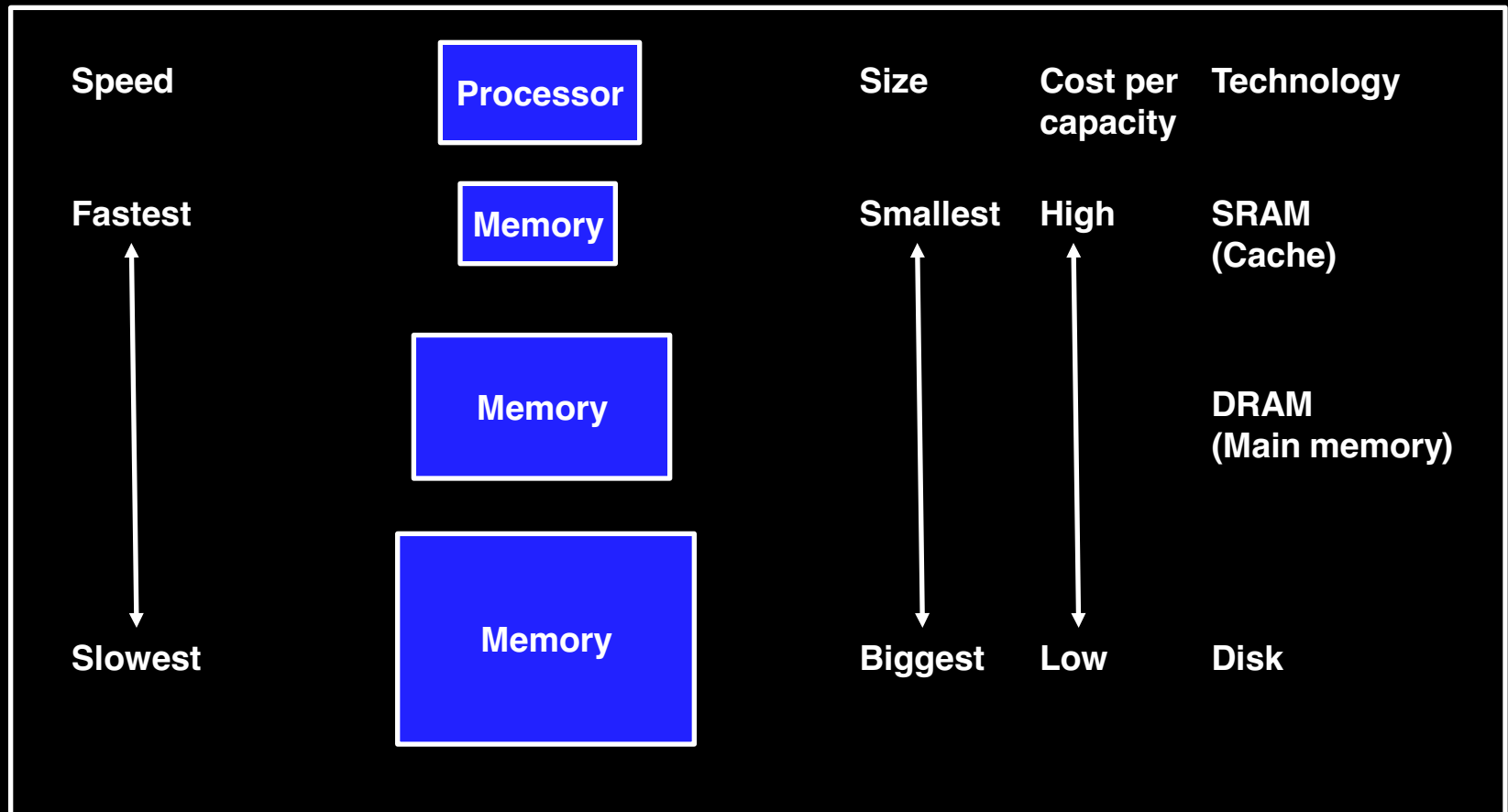
Outline

- **Introduction**
- **Shared Memory Multiprocessor**
- **Cache Coherence Problem**
- **Coherence Summary**

Locality

- **Temporal locality**
 - Refers to the reuse of specific data, or resources, within a relatively small time duration
- **Spatial locality**
 - Refers to the use of data elements within relatively close storage locations

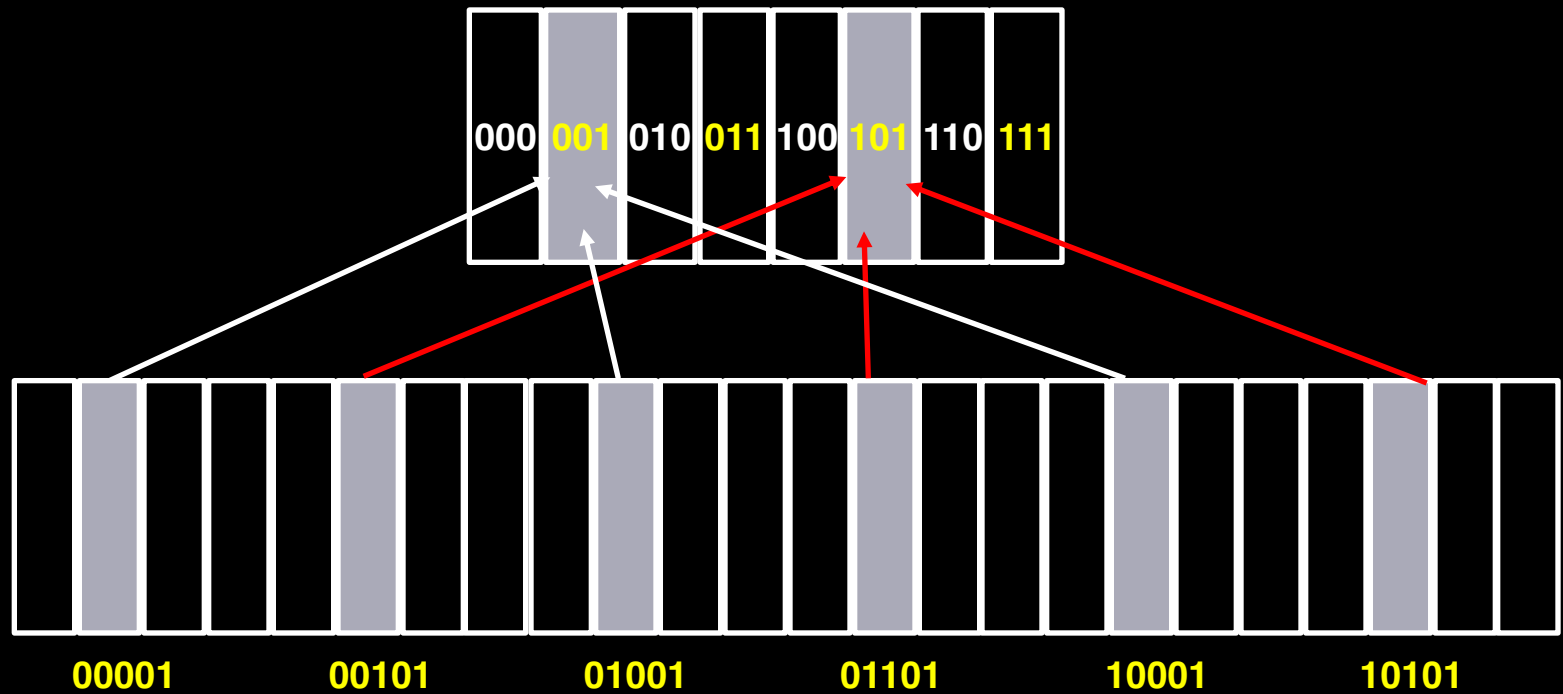
Memory Hierarchy



Cache Hit and Cache Miss

- **Cache hit**
 - When the processor finds a requested data in the cache
- **Cache miss**
 - When the processor does not find a data it needs in the cache
- **Hit rate**
 - The percentage of accesses that result in cache hit
- **Miss rate**
 - $1 - \text{Hit rate}$
- **Hit time**
 - The time to hit data in the cache
- **Miss penalty**
 - The time to retrieve the block from memory

Direct Mapped



$$\text{Index} = \log_2(8) = 3$$

Direct Mapped Example

22	10110 ₂	Miss	110 ₂
26	11010 ₂	Miss	010 ₂
22	10110 ₂	Hit	110 ₂
26	11010 ₂	Hit	010 ₂
16	10000 ₂	Miss	000 ₂
3	00011 ₂	Miss	011 ₂
16	10000 ₂	Hit	000 ₂
18	10010 ₂	Miss	010 ₂
16	10000 ₂	Hit	000 ₂

Index	V	Tag	Data
000	Y	10 ₂	Memory(10000 ₂)
001	N		
010	Y	10 ₂	Memory(10010 ₂)
011	Y	00 ₂	Memory(00011 ₂)
100	N		
101	N		
110	Y	10 ₂	Memory(10110 ₂)
111	N		

Writing Policies

- **Write-through**
 - Write is done synchronously both to the cache and to the backing store
- **Write-back**
 - Initially, writing is done only to the cache
 - The write to backing store is postponed until the cache blocks containing the data are about to be replaced by new content

Reduce Cache Miss

- **Fully associative**
 - A main memory block can be placed in any of the cache slots
- **N-way set associative**
 - Each location in main memory can be cached in either of N locations in the cache

Direct-mapped Cache

- Block address: 0, 8, 0, 6, 8
- 4 blocks

Block address	Index
0	0
6	2
8	0

	5 miss	0	1	2	3
0	Miss	Memory(0)			
8	Miss	Memory(8)			
0	Miss	Memory(0)			
6	Miss	Memory(0)		Memory(6)	
8	Miss	Memory(8)		Memory(6)	

N-way Set Associative Cache

- Block address: 0, 8, 0, 6, 8
- 4 blocks

Block address	Index
0	0
6	0
8	0

	4 miss	Set 0	Set 0	Set 1	Set 1
0	Miss	Memory(0)			
8	Miss	Memory(0)	Memory(8)		
0	Hit	Memory(0)	Memory(8)		
6	Miss	Memory(0)	Memory(6)		
8	Miss	Memory(8)	Memory(6)		

Fully Associative Cache

- Block address: 0, 8, 0, 6, 8
- 4 blocks

	3 miss	0	1	2	3
0	Miss	Memory(0)			
8	Miss	Memory(0)	Memory(8)		
0	Hit	Memory(0)	Memory(8)		
6	Miss	Memory(0)	Memory(8)	Memory(6)	
8	Hit	Memory(8)	Memory(8)	Memory(6)	

Replacement Policies

- In order to make room for the new entry on a cache miss, the cache may have to evict one of the existing entries
- One popular replacement policy, least-recently used (LRU), replaces the least recently accessed entry

Multi-level Cache

- **Larger caches have better hit rates but long latency**
- **Multi-level caches generally operate by checking the smallest level 1 (L1) cache first**
 - Hit: the processor proceeds at high speed
 - Miss: the next level cache (L2) is checked

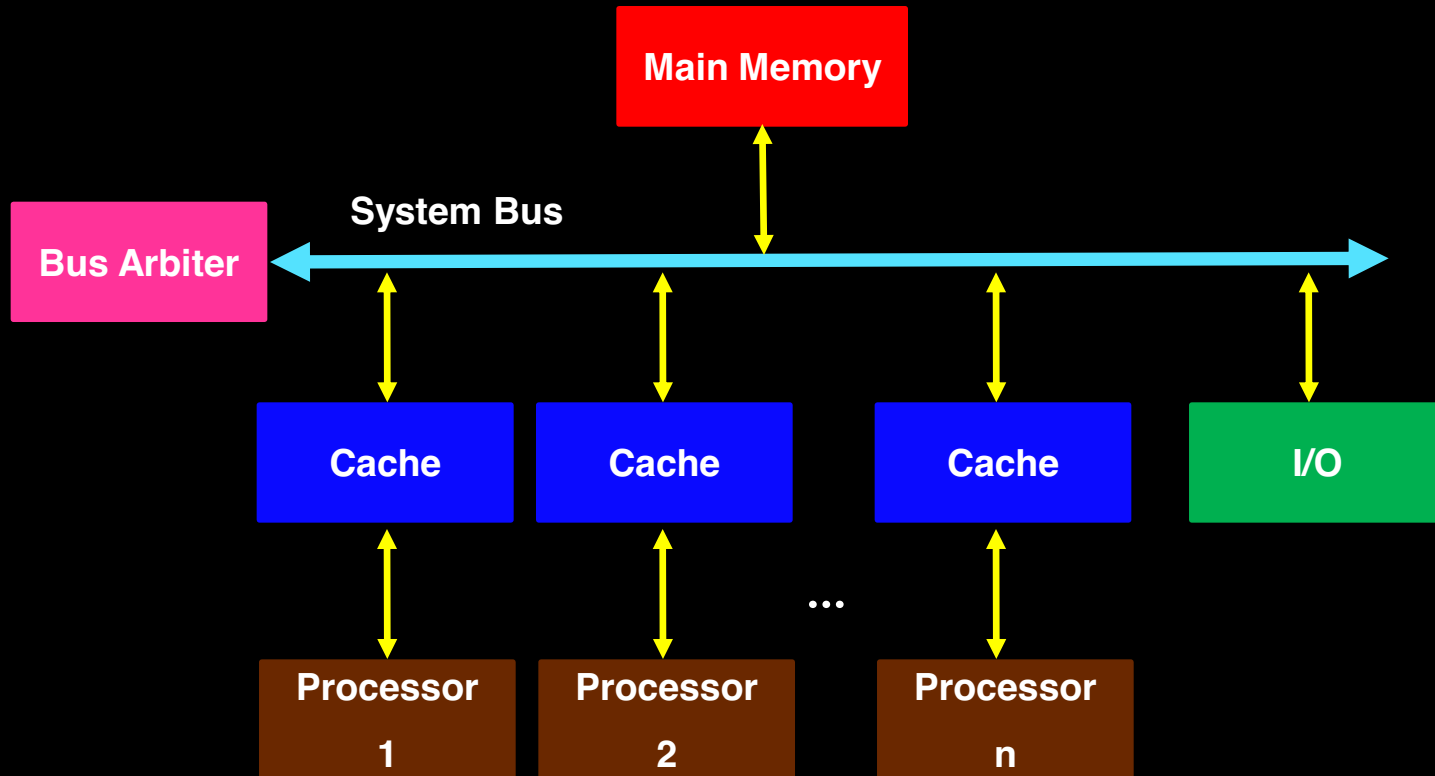
Outline

- Introduction
- **Shared Memory Multiprocessor**
- Cache Coherence Problem
- Coherence Summary

Shared Memory Multiprocessor

- **Symmetric multiprocessing(SMP)**
- **Two or more identical processors are connected to a single shared main memory**
- **Uniform memory access(UMA)**
 - An uniform access time from any processor
- **Nonuniform memory access(NUMA)**
 - The access time depends on the location of a data in memory

Shared Memory Multiprocessor



Outline

- Introduction
- Shared Memory Multiprocessor
- **Cache Coherence Problem**
- Coherence Summary

Cache Coherence Problem

- Different processors may have different values for the same location in local caches

Time	Event	CPU A cache content	CPU B cache content	X memory content
0				0
1	CPU A loads X	0		0
2	CPU B loads X	0	0	0
3	CPU A stores 1 to X	1	0	1

Cache Coherence Mechanisms

- **Directory-based coherence**
 - The data being shared is placed in a common directory that maintains the coherence between caches
- **Snooping**
 - Each cache monitors bus reads/writes

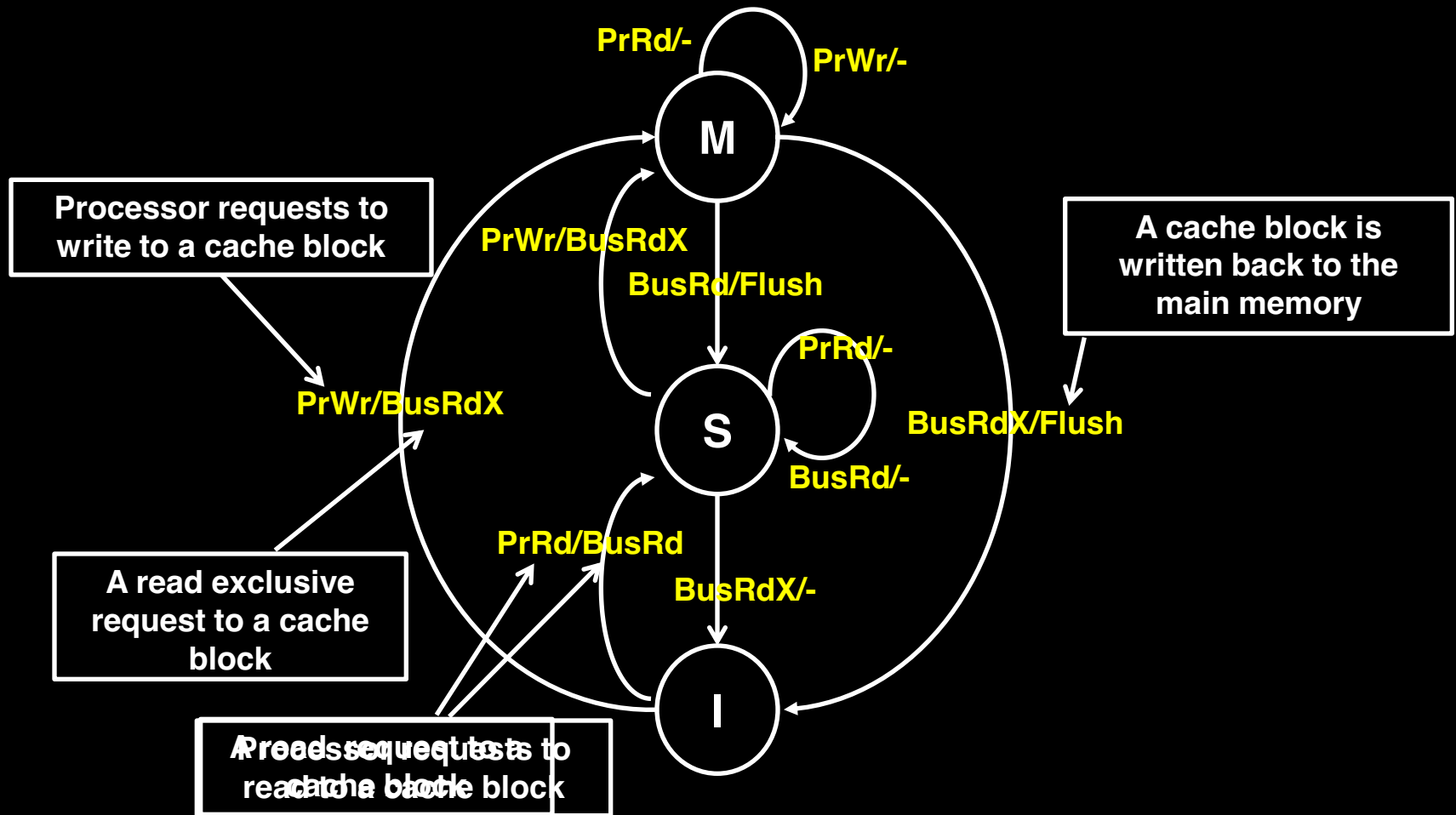
Coherence Protocol

- **Protocol maintains the consistency between all the caches in a system of distributed shared memory**
- **Coherence protocol**
 - MSI protocol
 - MESI protocol
 - MESIF protocol
 - MOESI protocol

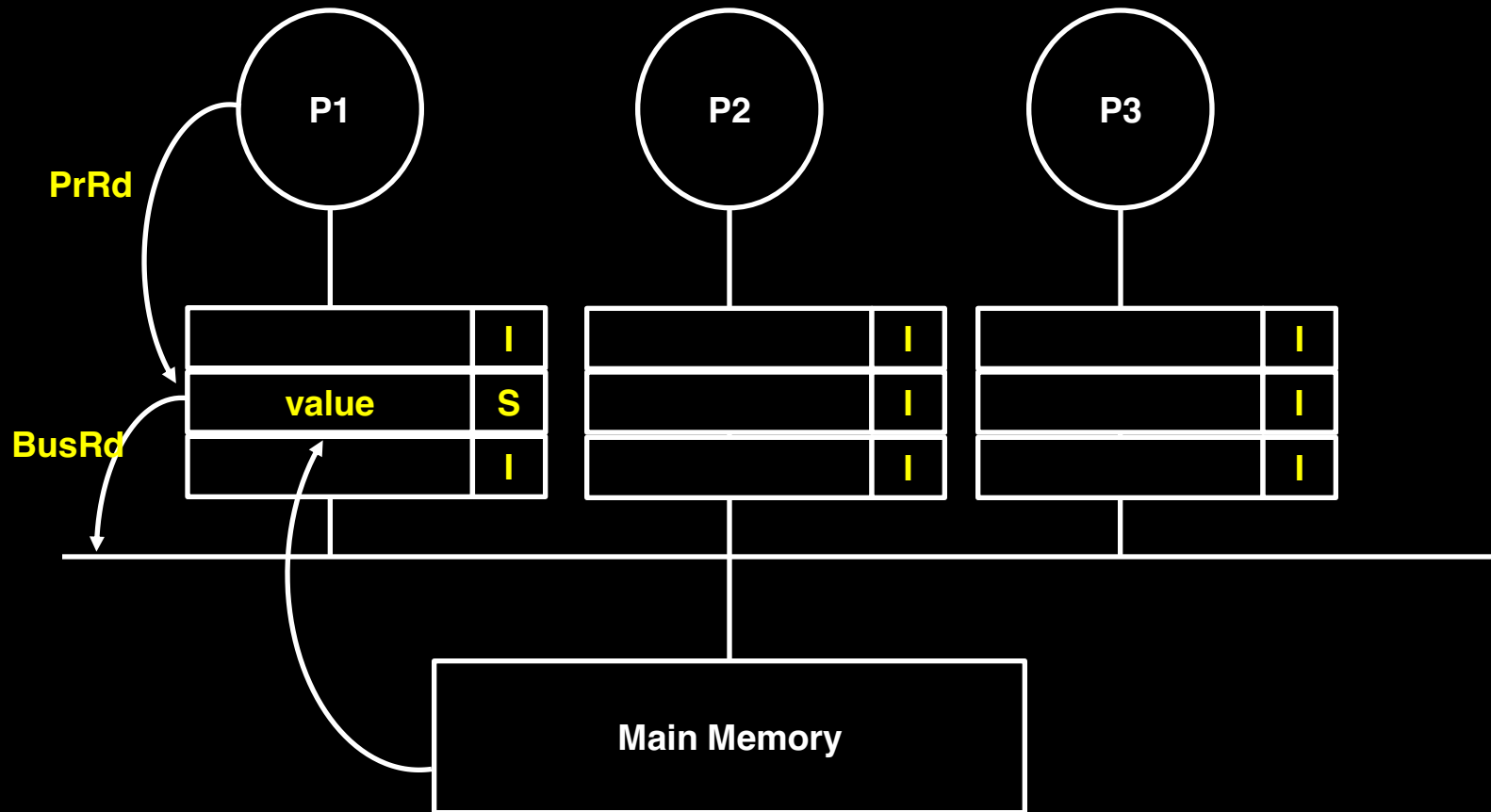
MSI Protocol

- **Three possible state**
 - Modified: The block has been modified in the cache
 - Shared: This block is unmodified and exists in at least one cache
 - Invalid: This block is invalid

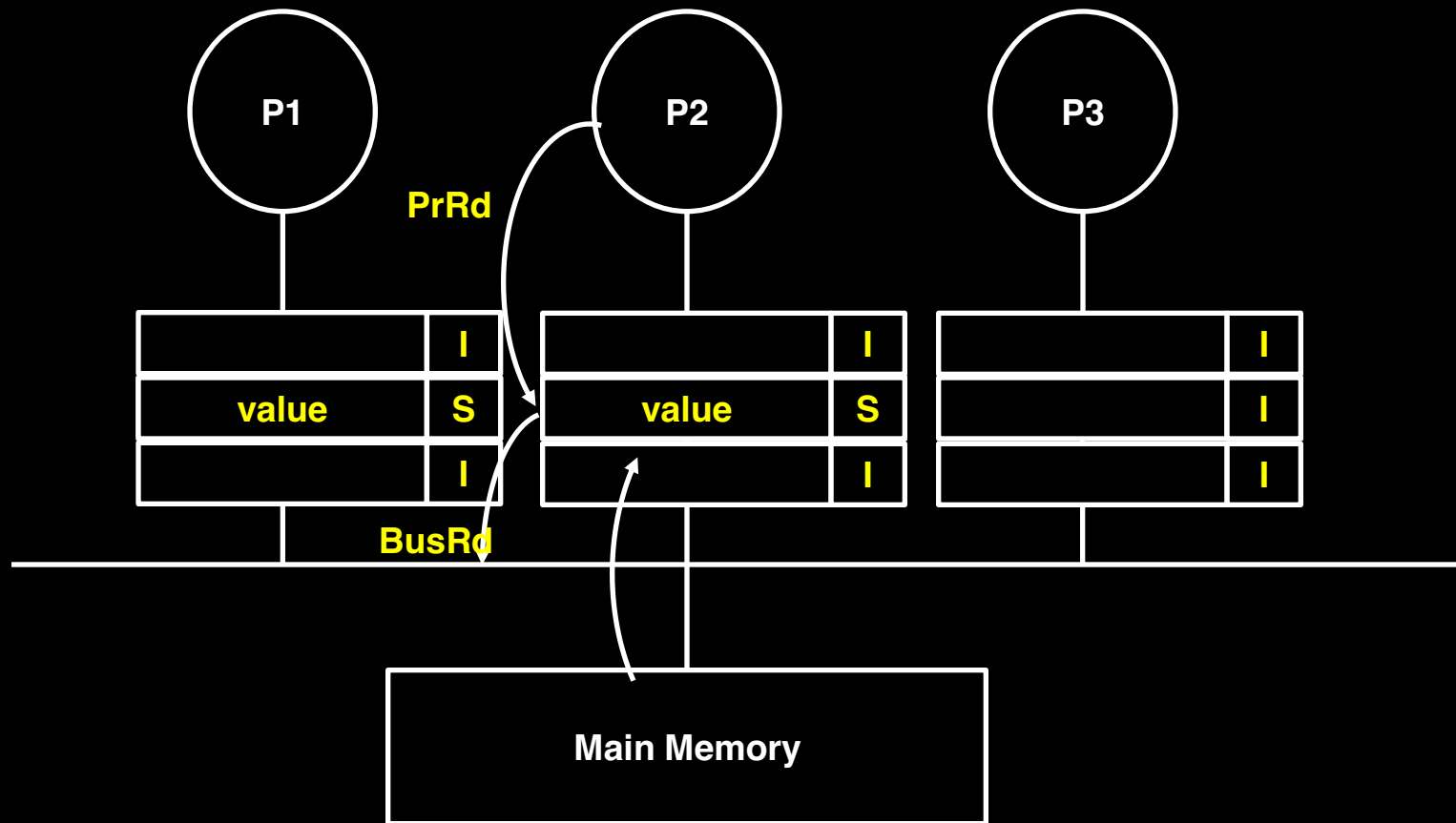
State Diagram



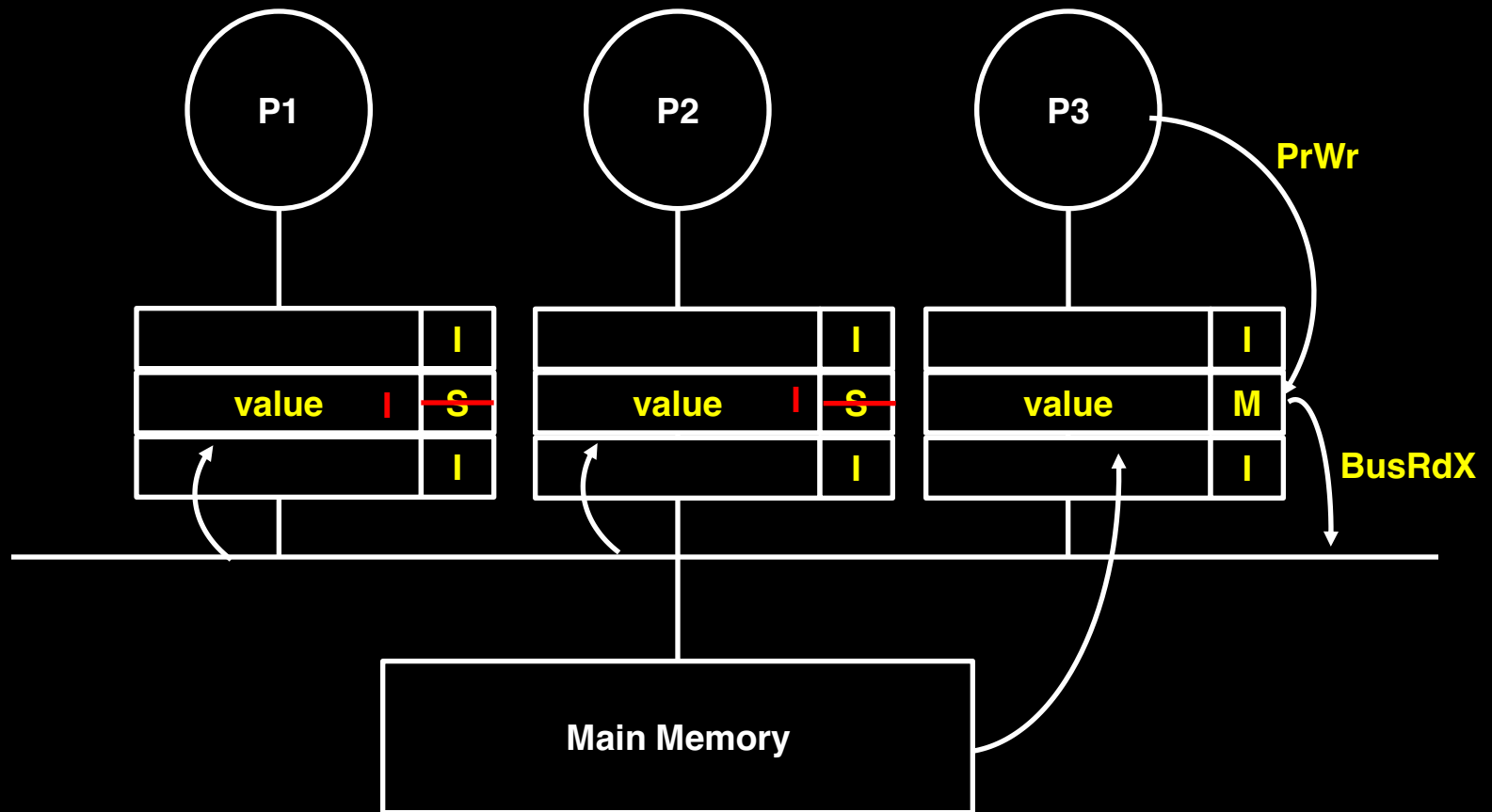
MSI Protocol Example



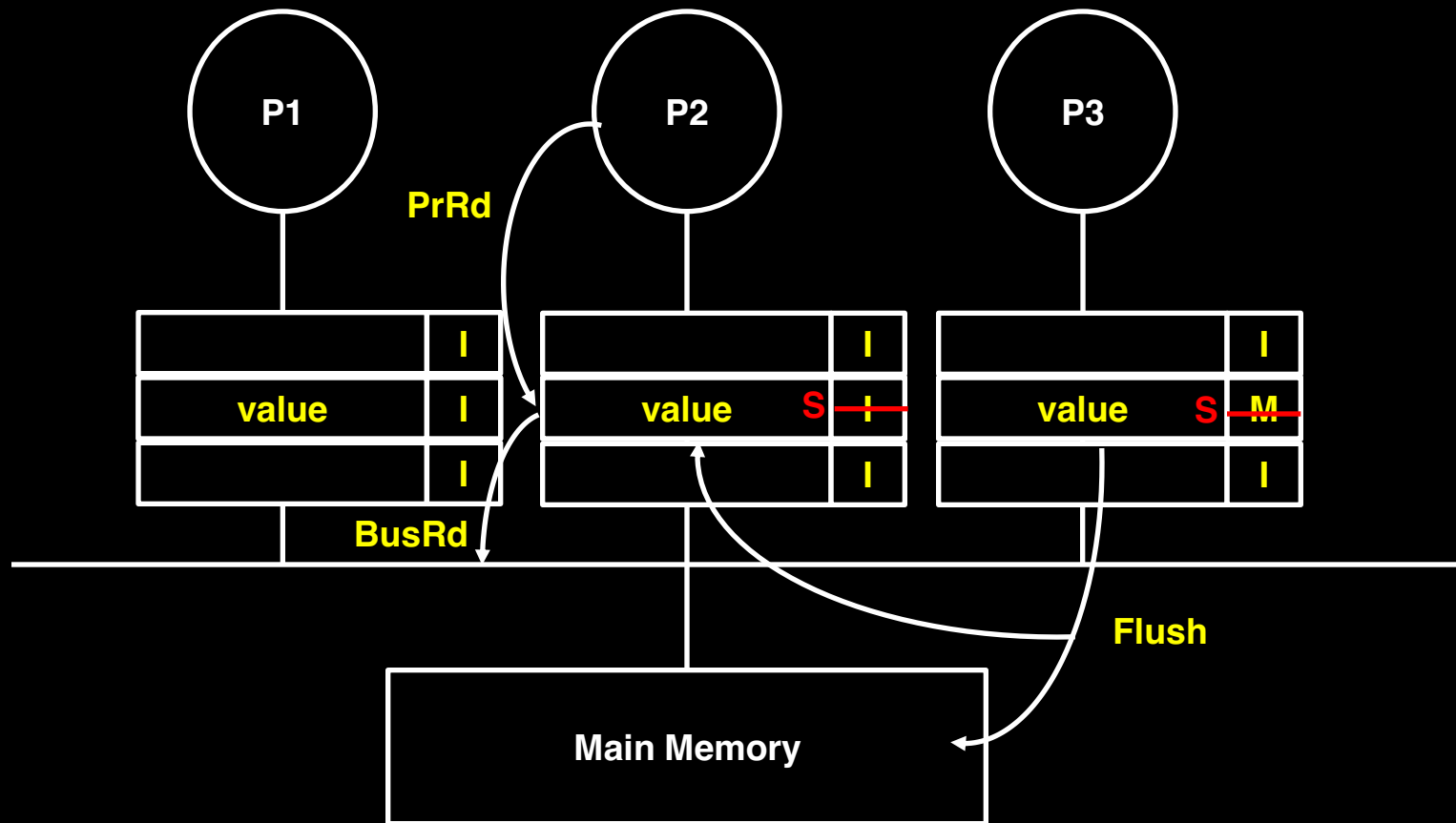
MSI Protocol Example



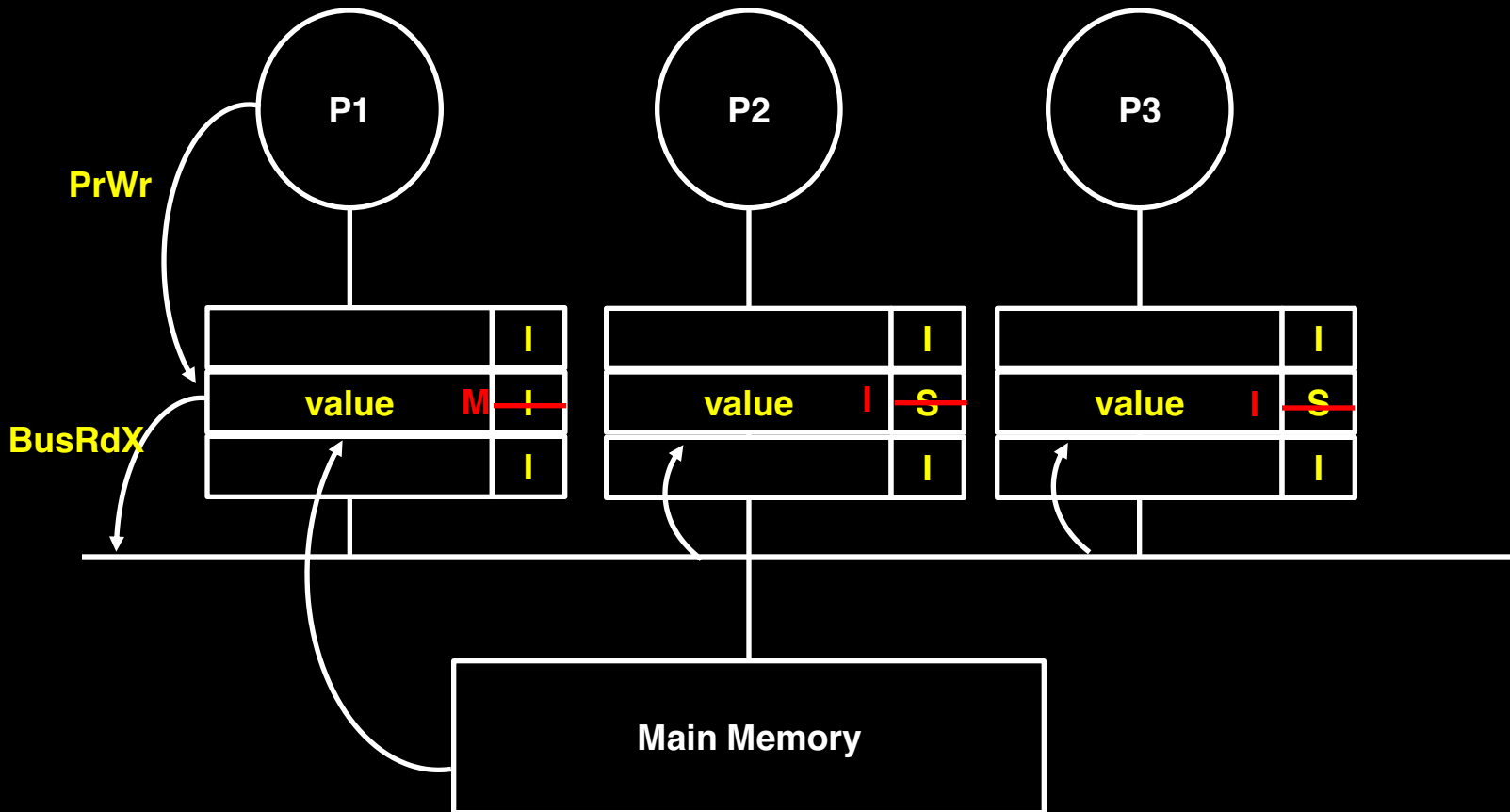
MSI Protocol Example



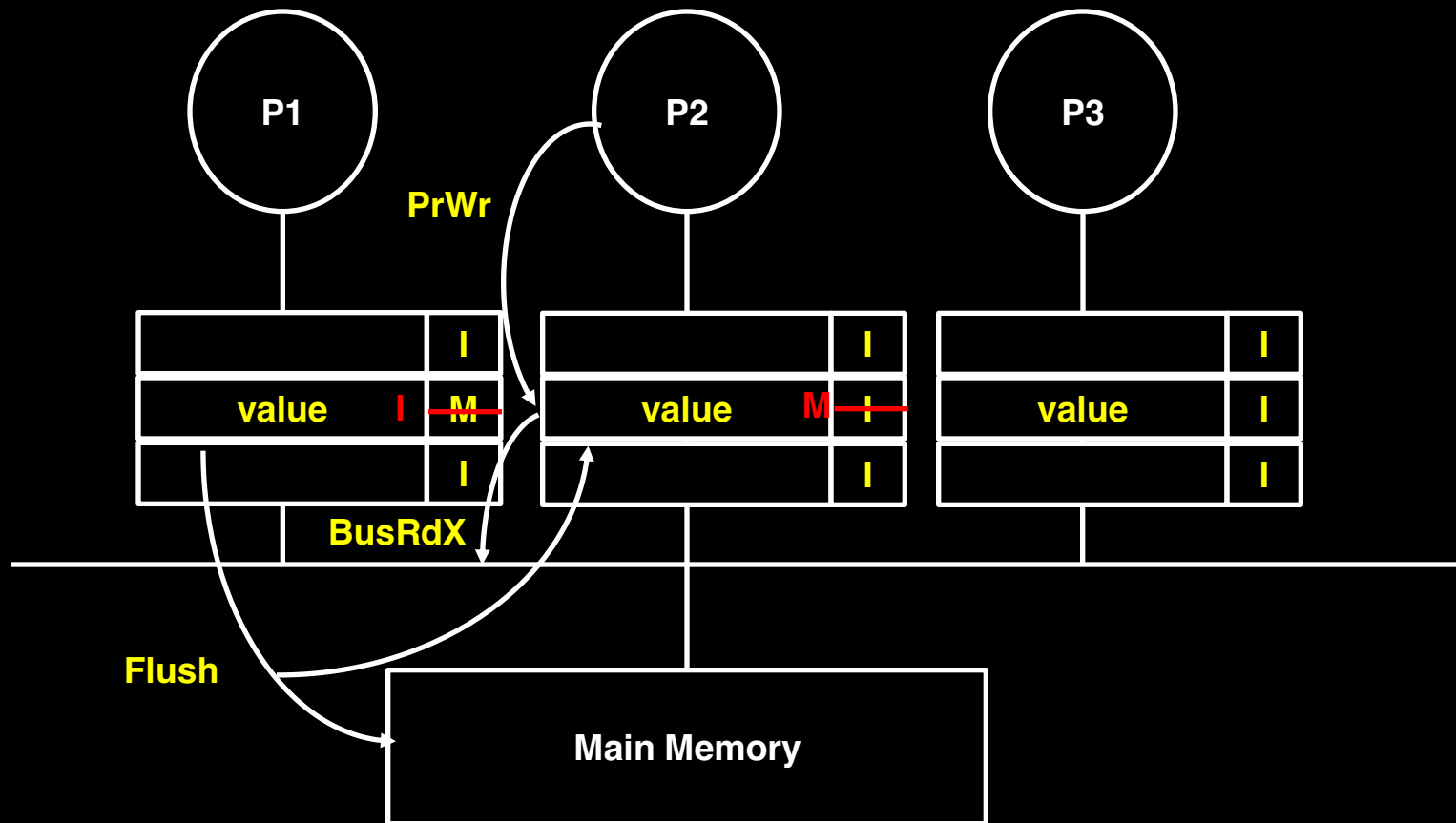
MSI Protocol Example



MSI Protocol Example



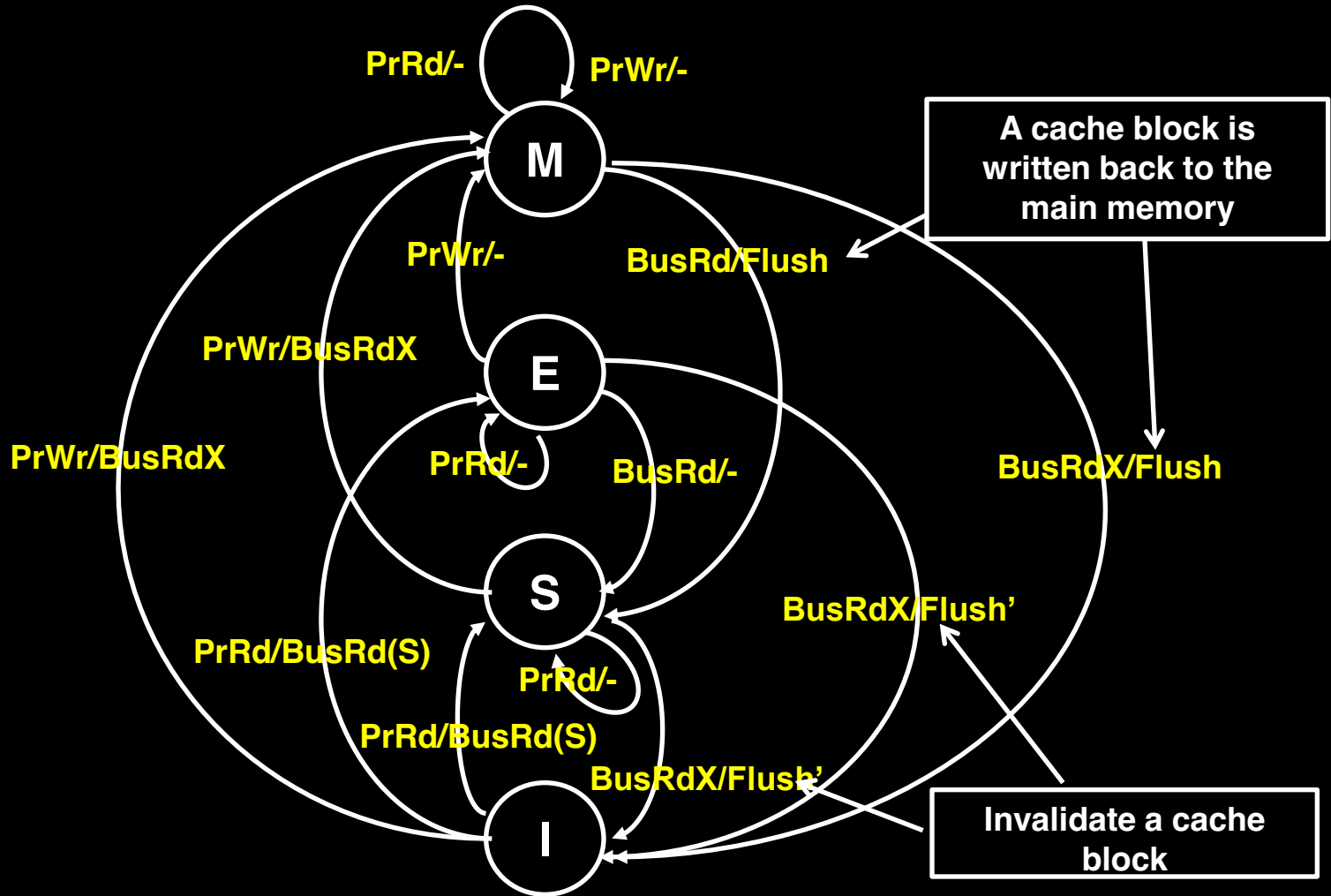
MSI Protocol Example



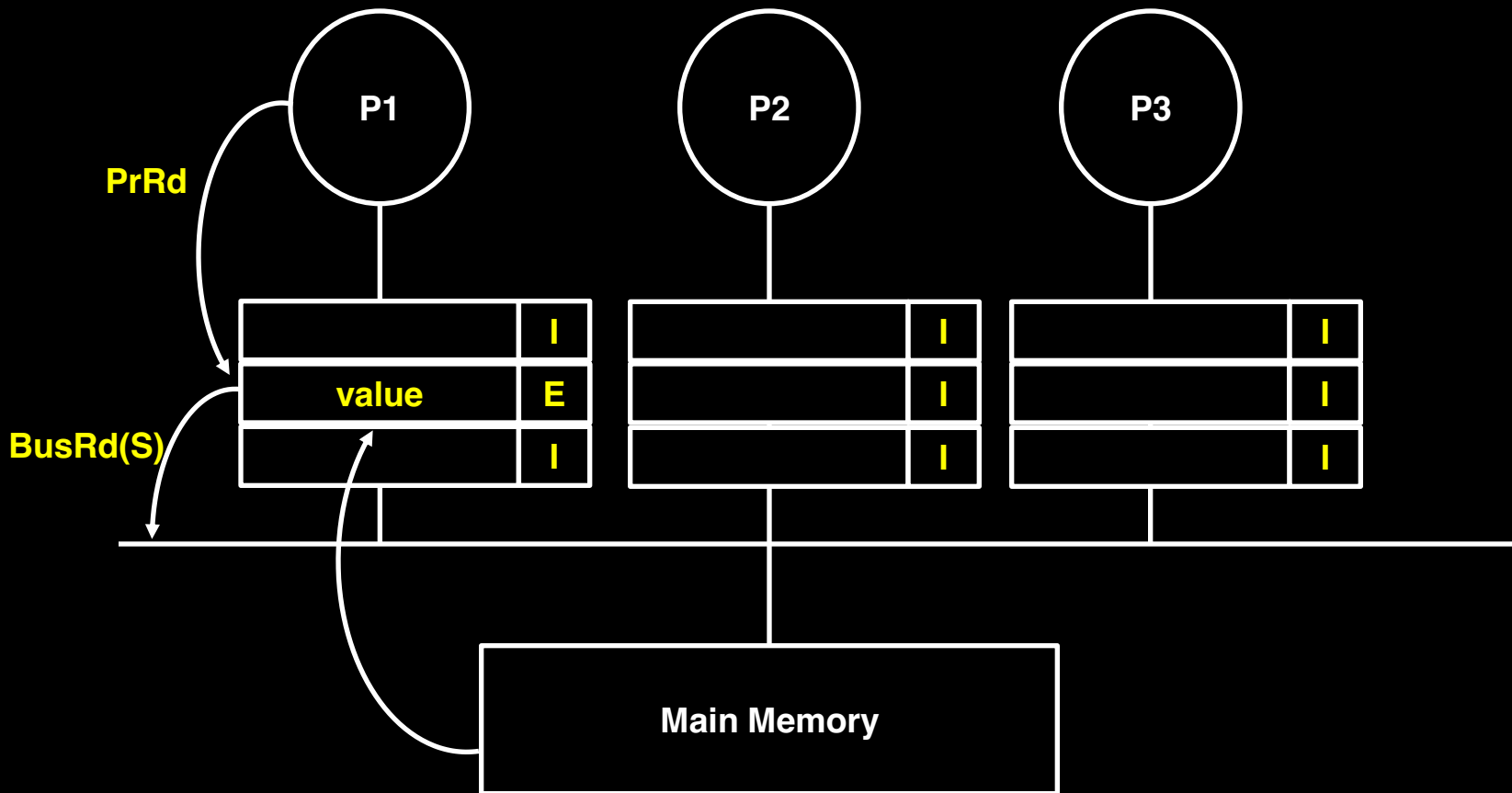
MESI Protocol

- **Four possible state**
 - Modified: The block has been modified in the cache
 - Exclusive: The cache line is present only in the current cache
 - Shared: This block is unmodified and exists in at least one cache
 - Invalid: This block is invalid

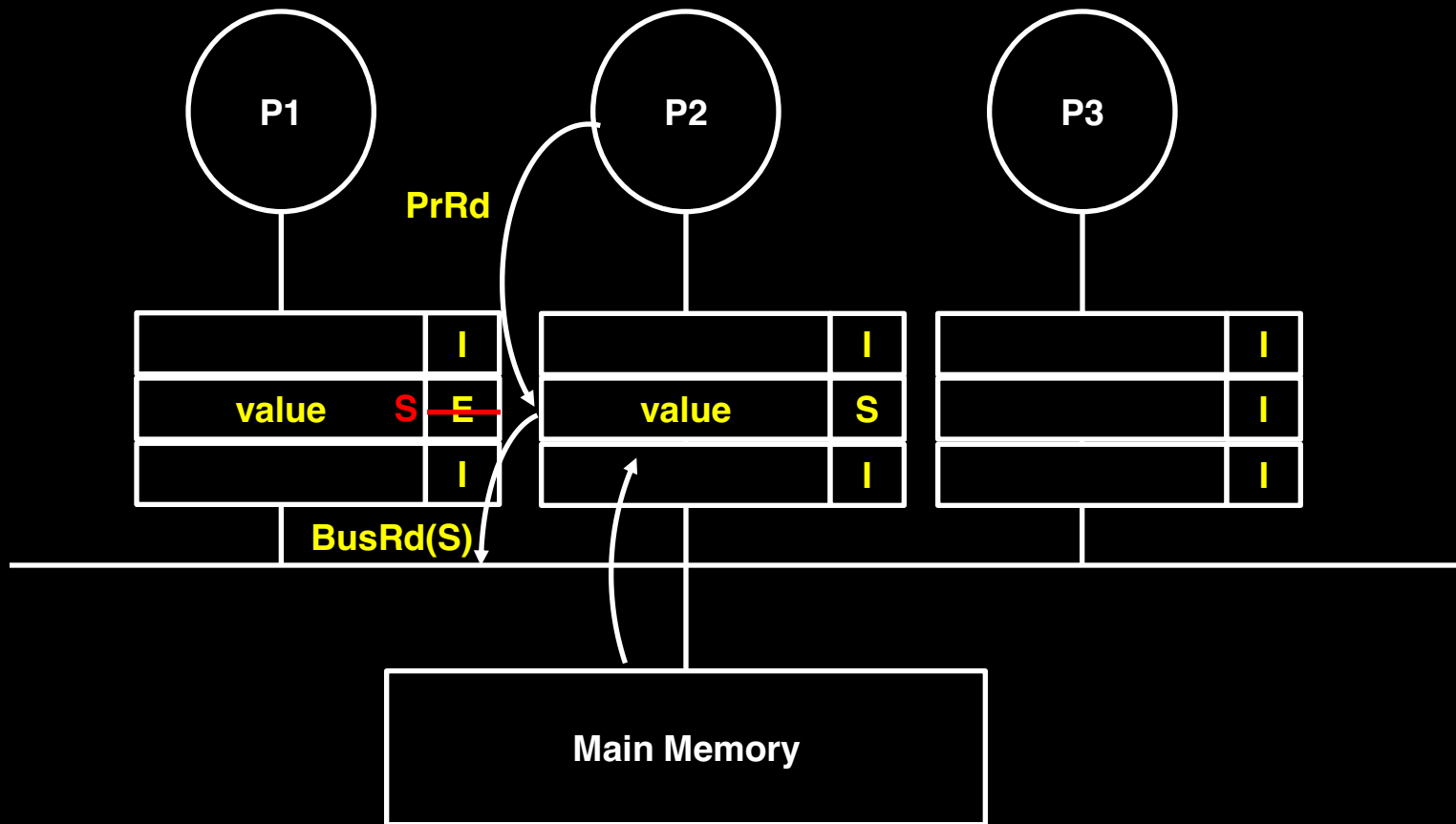
State Diagram



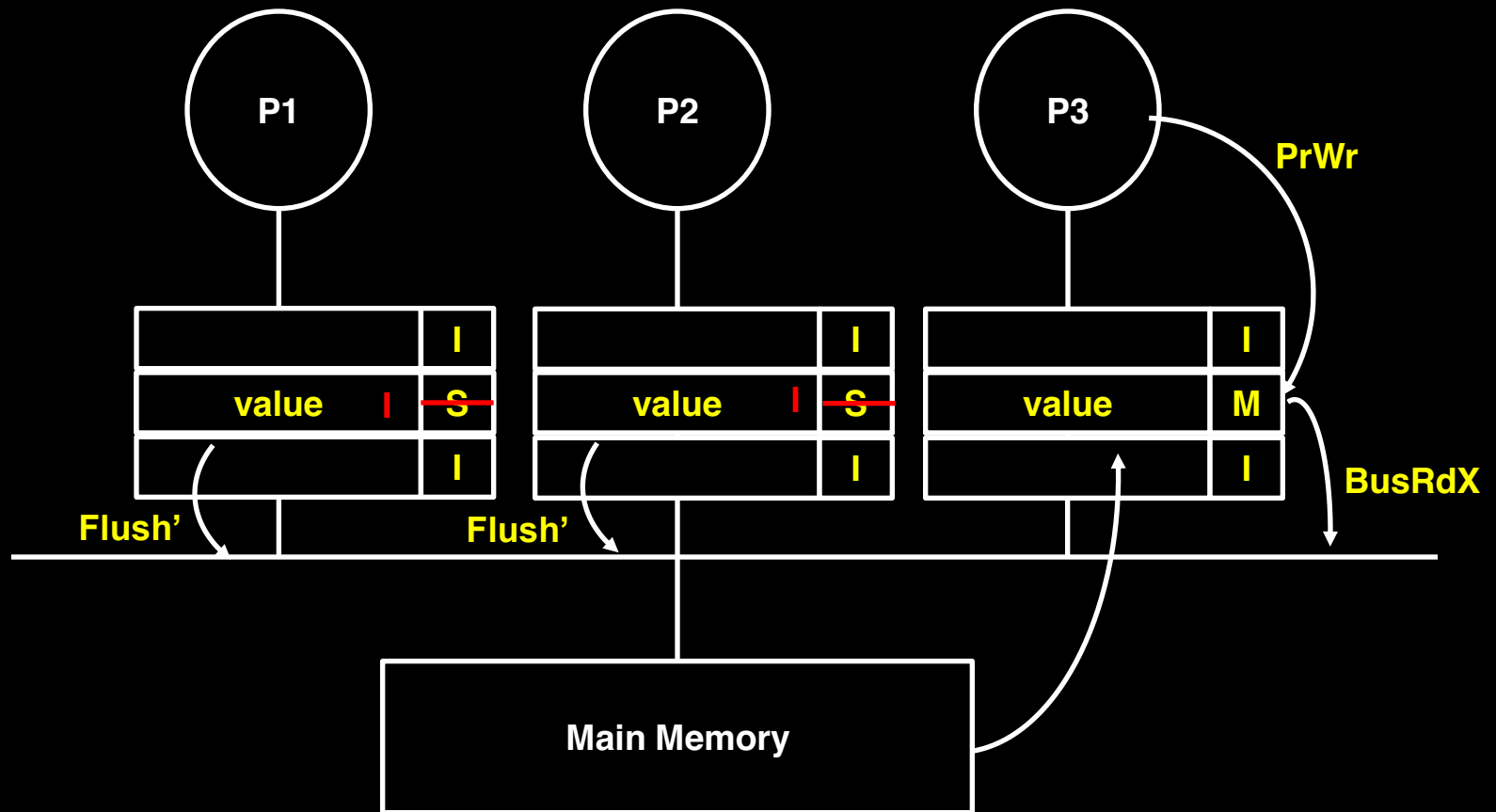
MESI Protocol Example



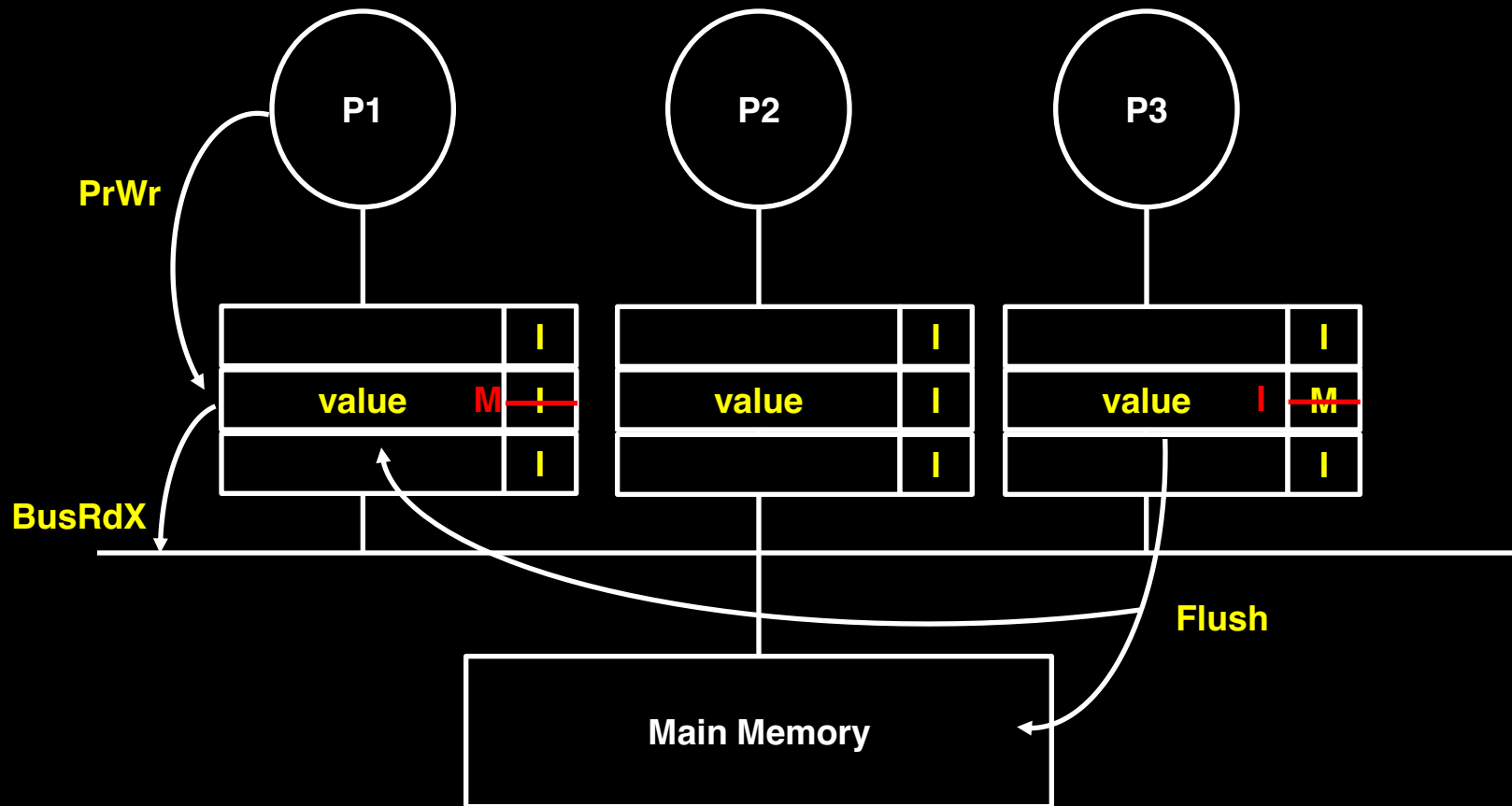
MESI Protocol Example



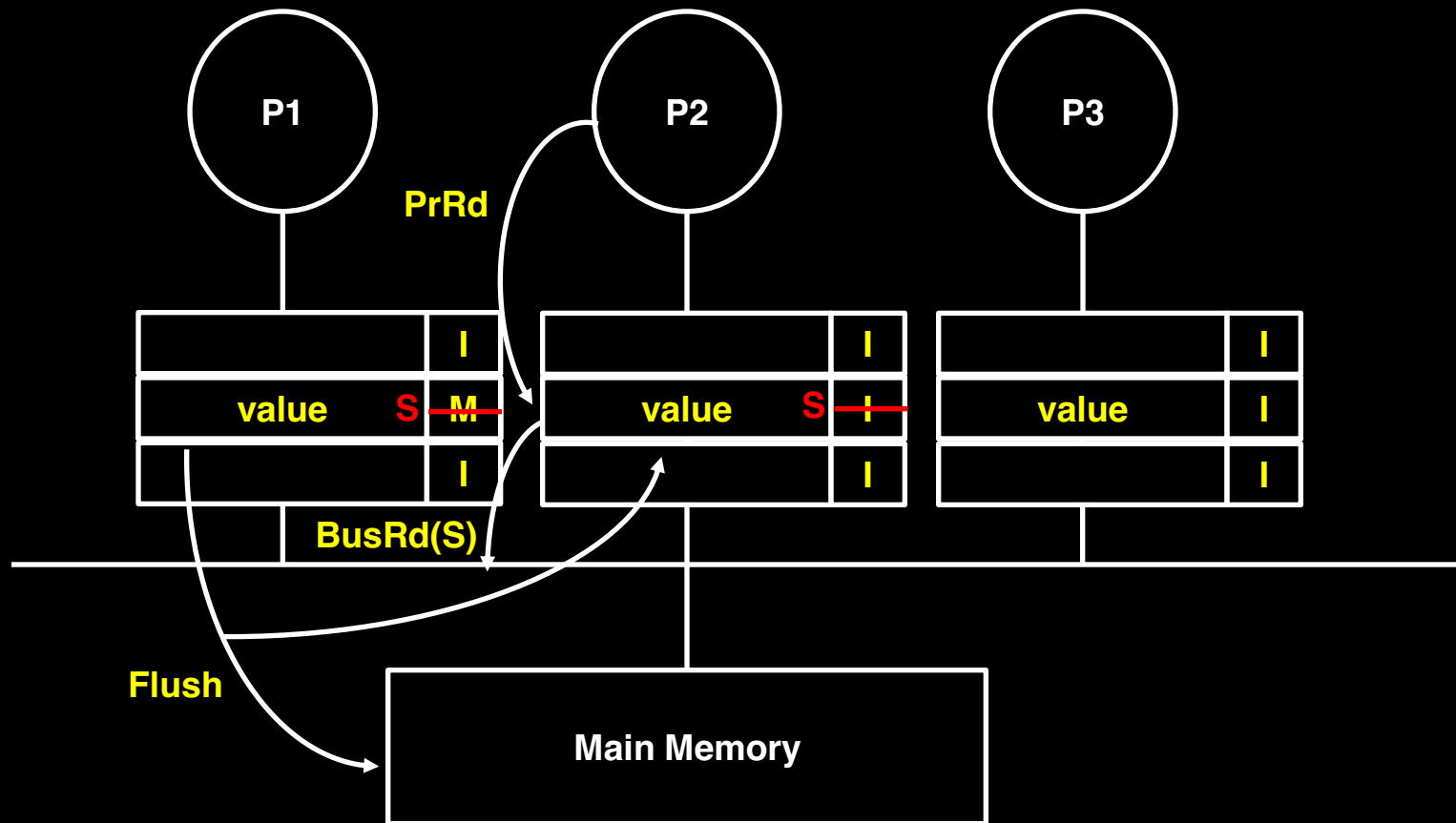
MESI Protocol Example



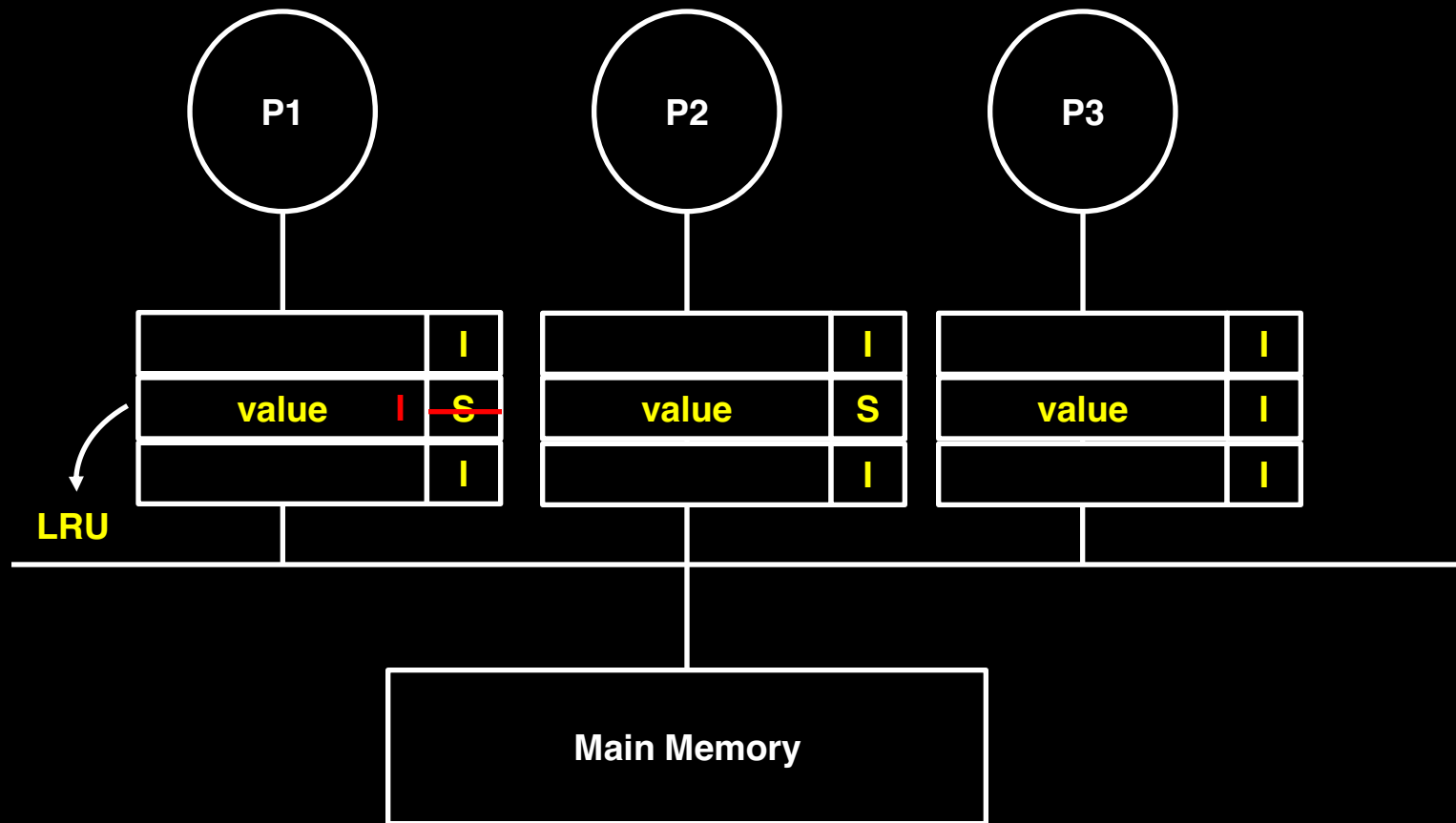
MESI Protocol Example



MESI Protocol Example



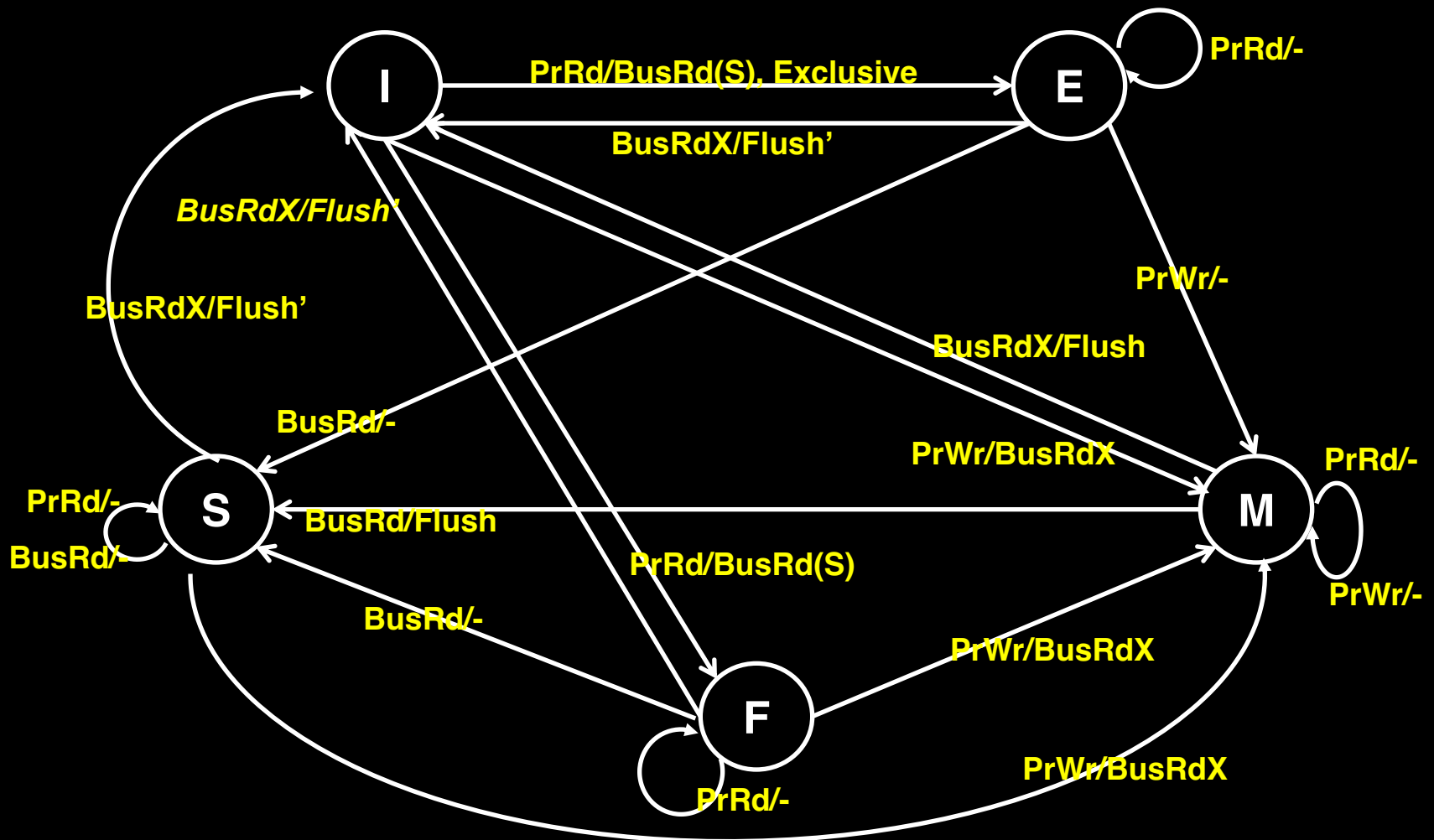
MESI Protocol Example



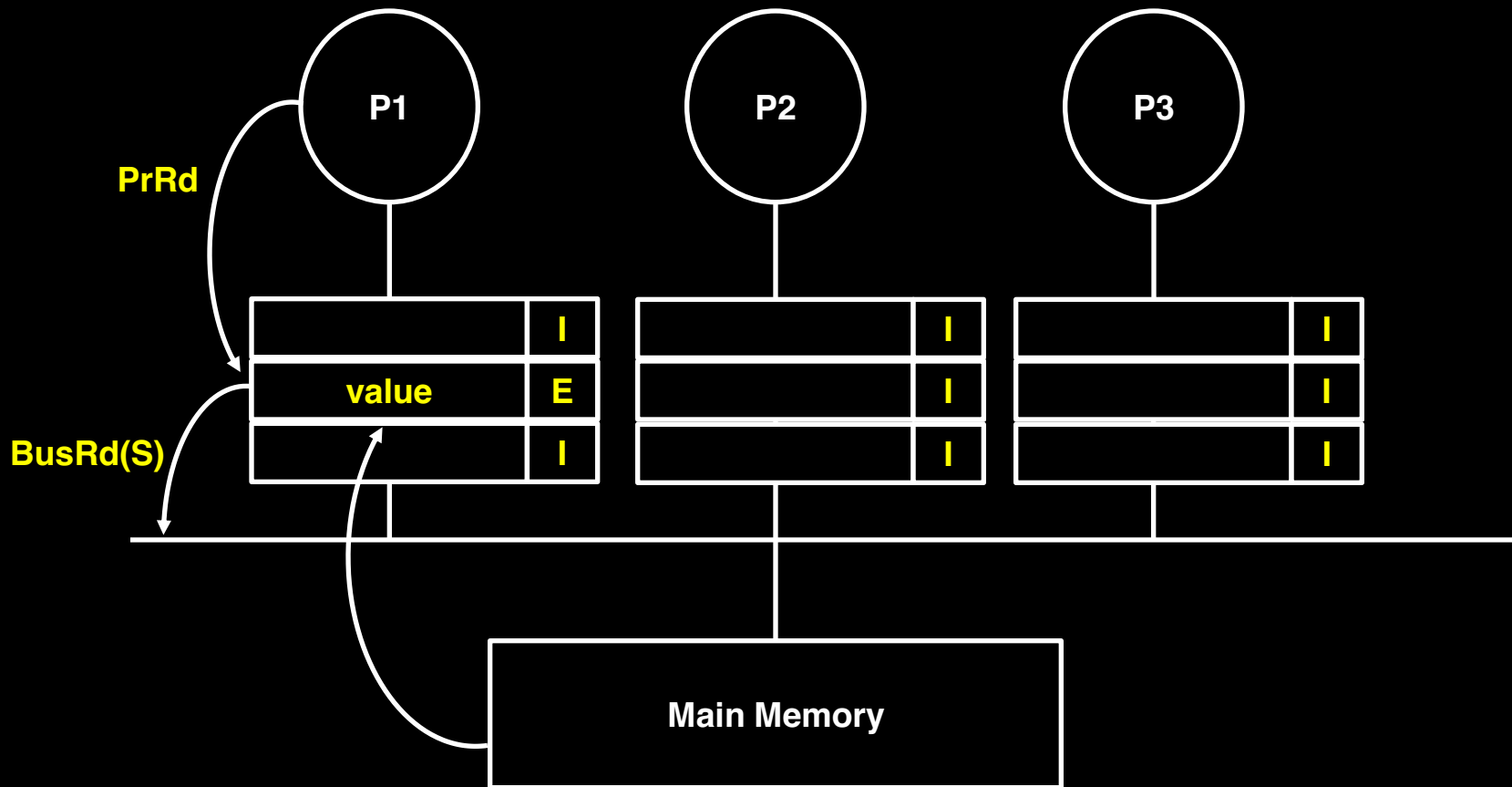
MESIF Protocol

- **Five possible state**
 - Modified
 - Exclusive
 - Shared
 - Invalid
 - Forward
- **The M, E, S and I states are the same as in the MESI protocol**
- **The most recent requestor of a line is assigned the F state; when a cache in the F state responds, it gives up the F state to the new cache**

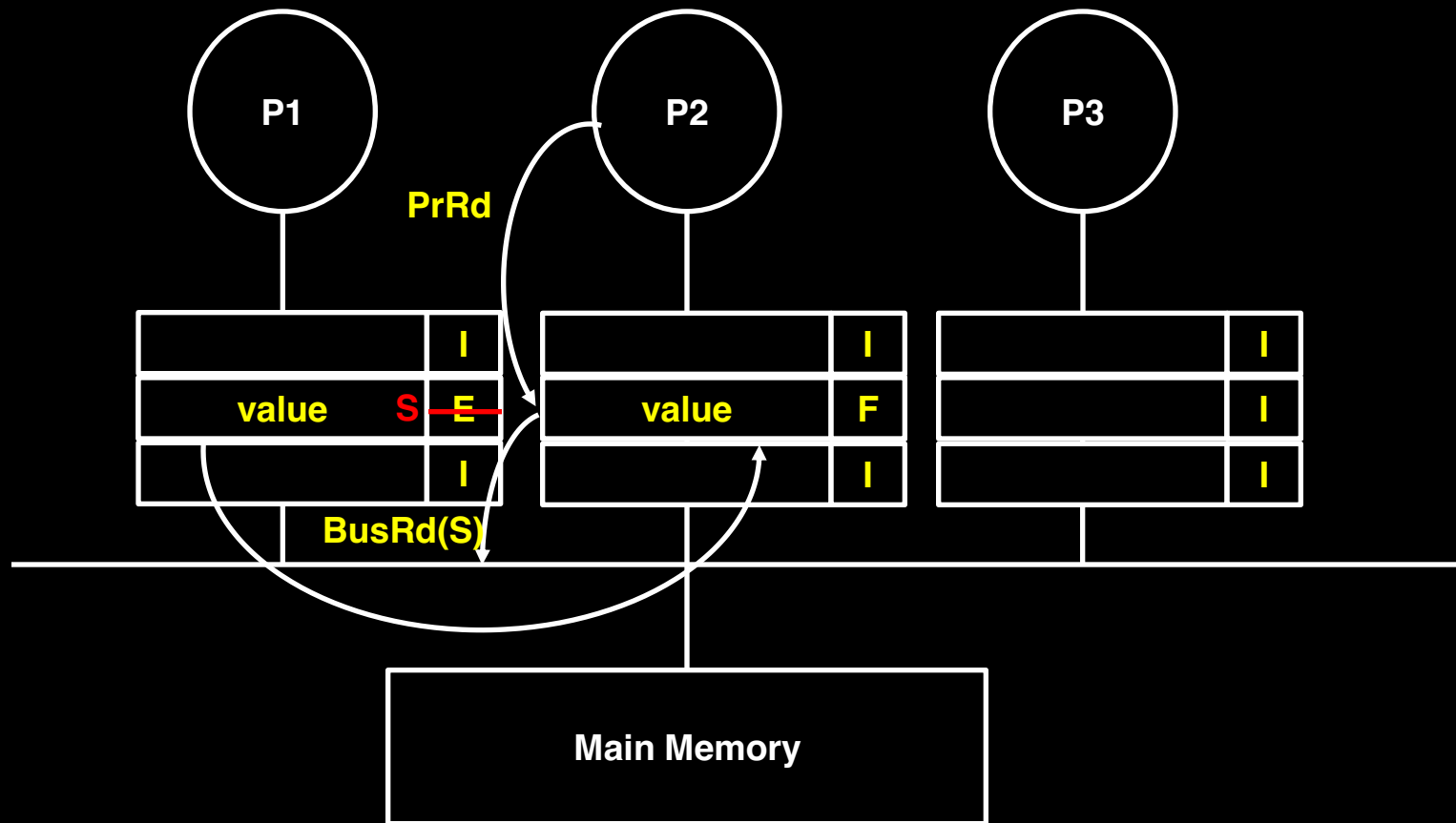
State Diagram



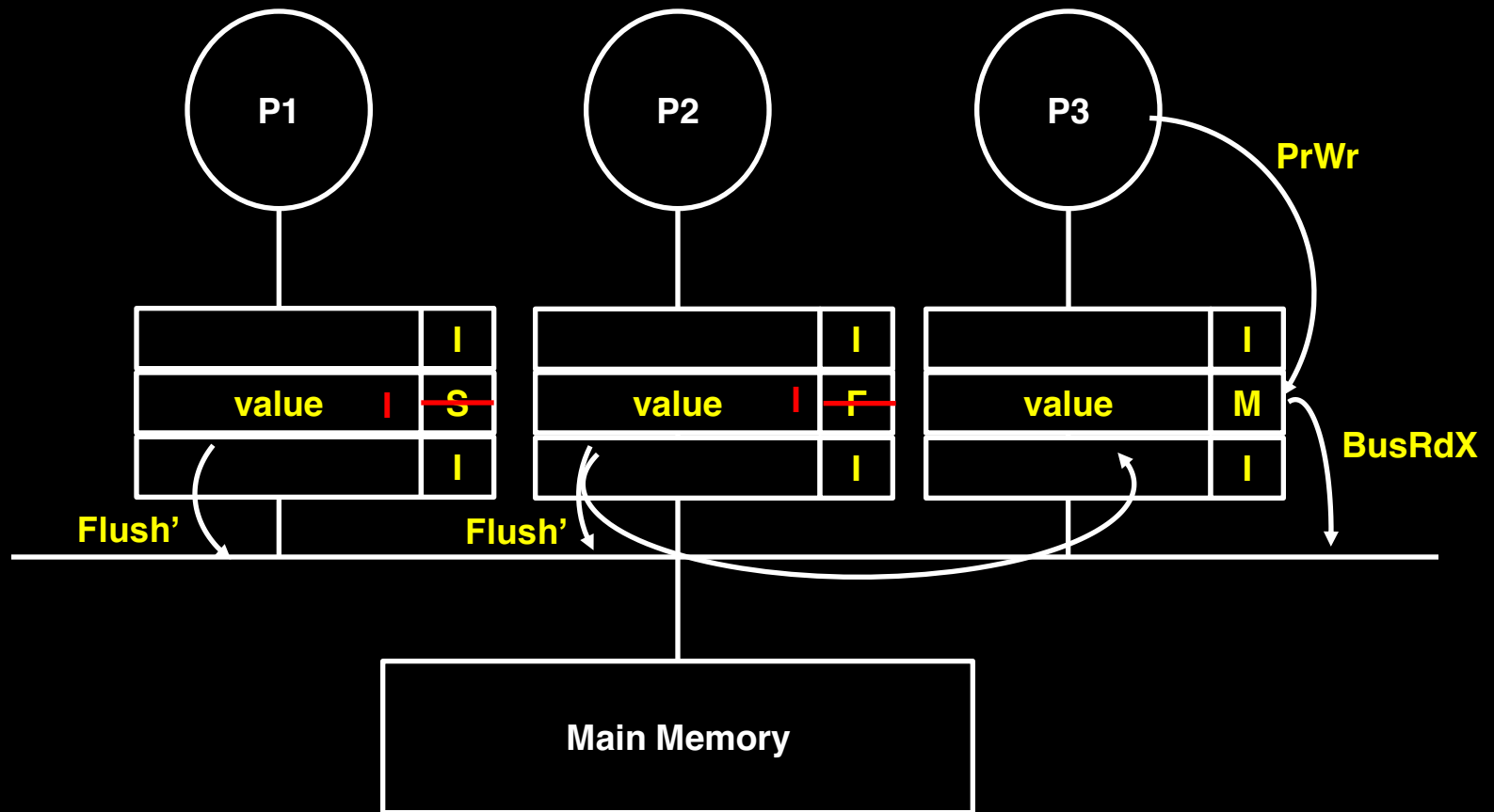
MESIF Protocol Example



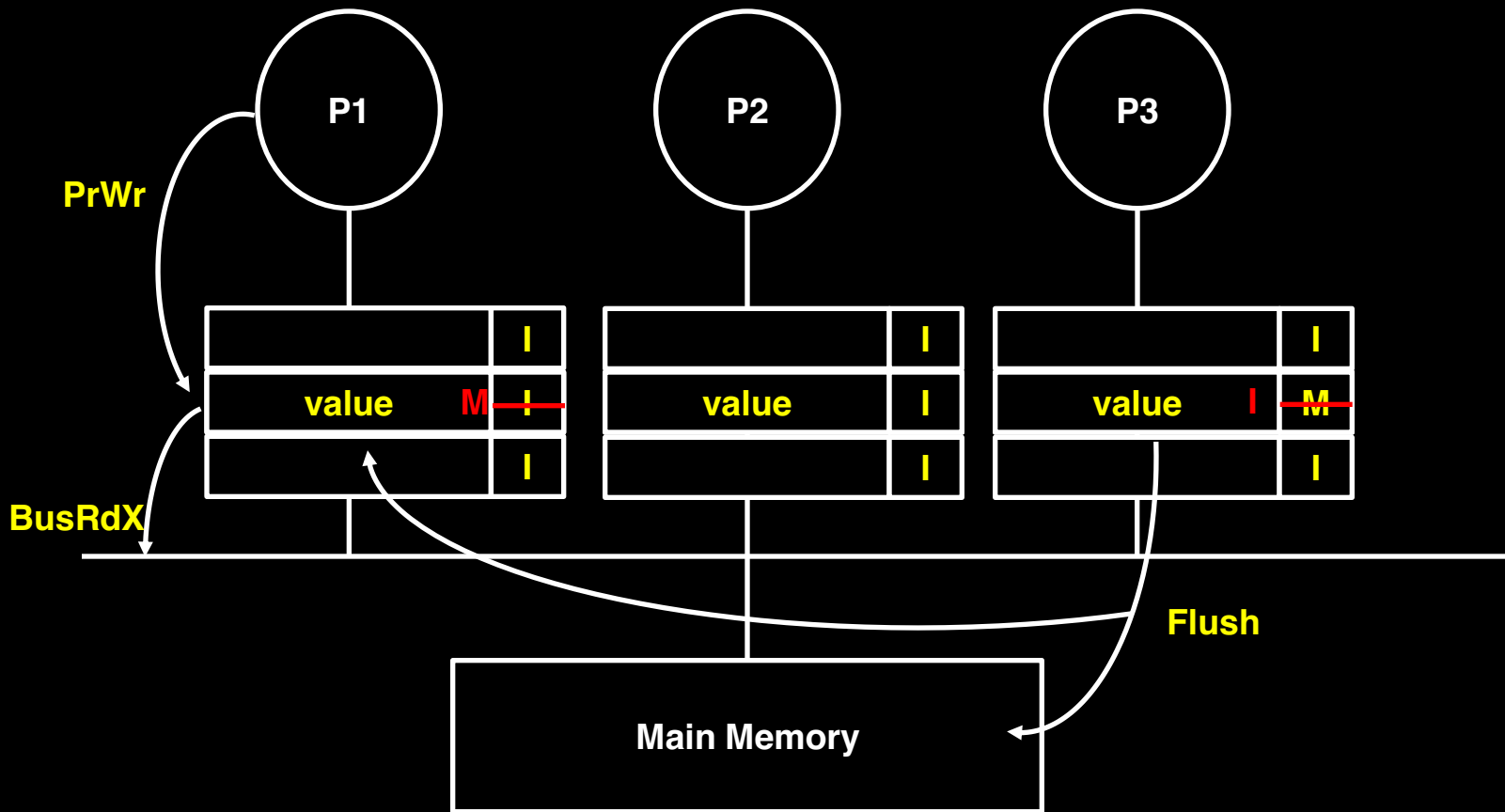
MESIF Protocol Example



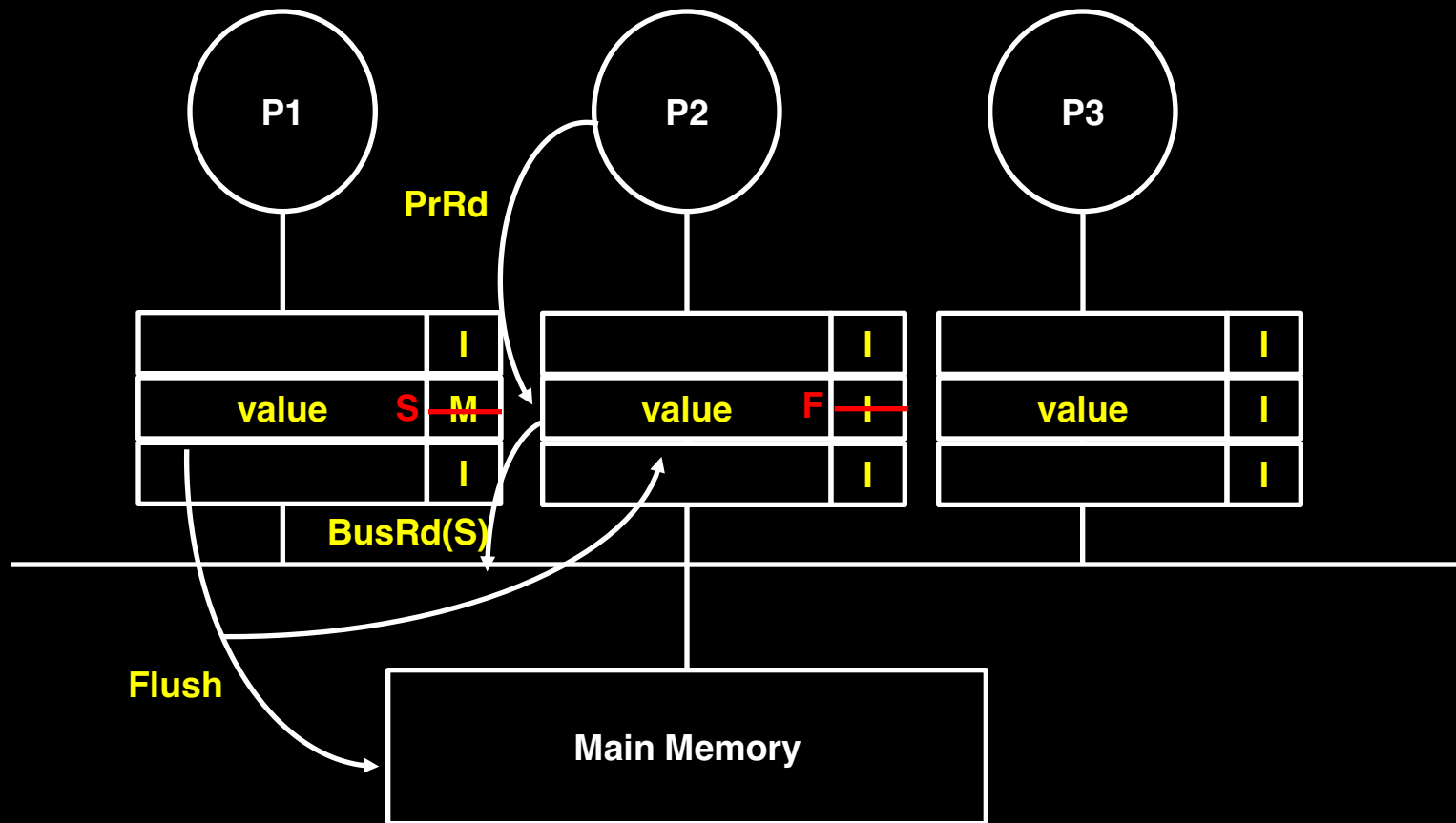
MESIF Protocol Example



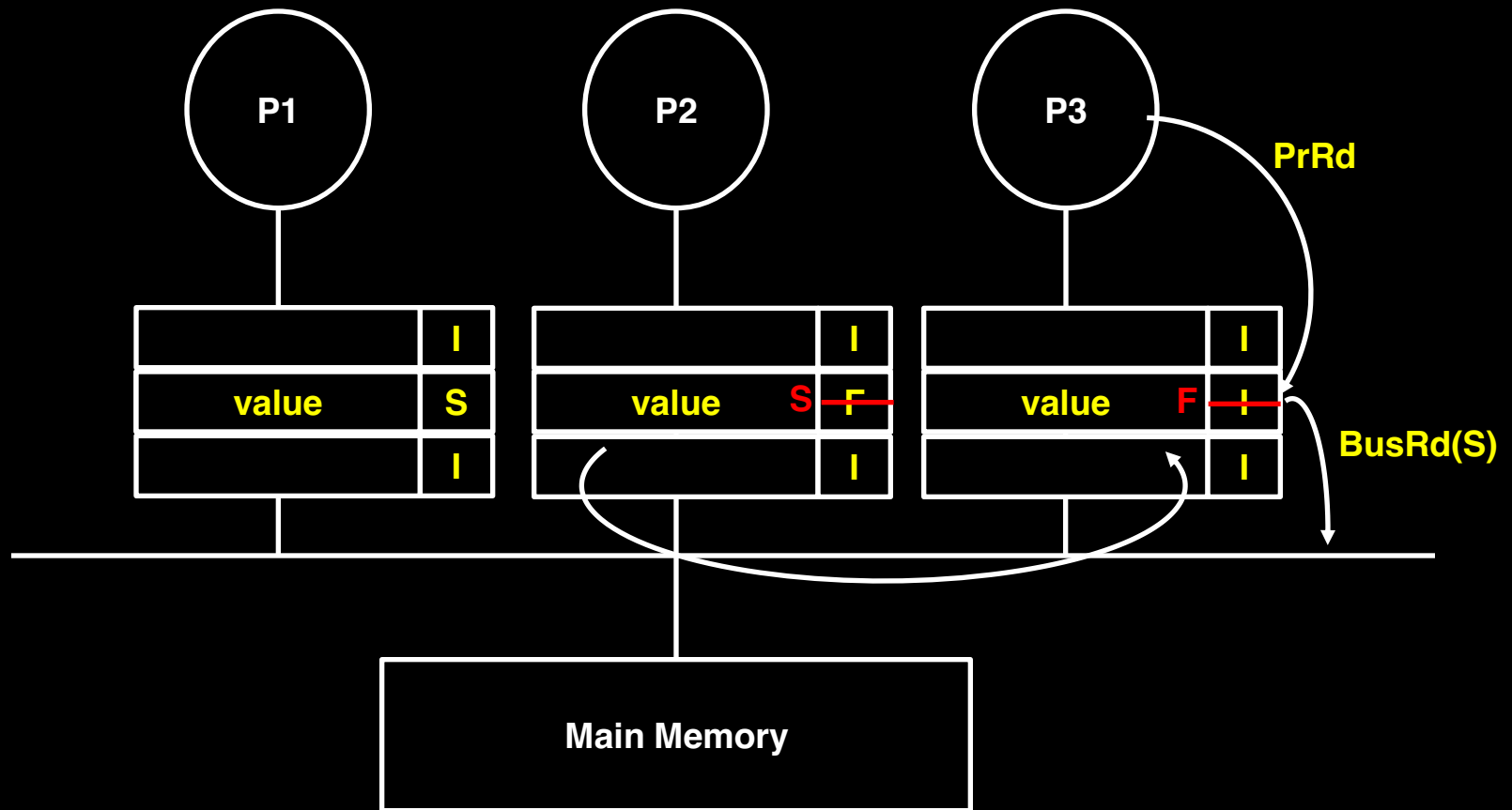
MESIF Protocol Example



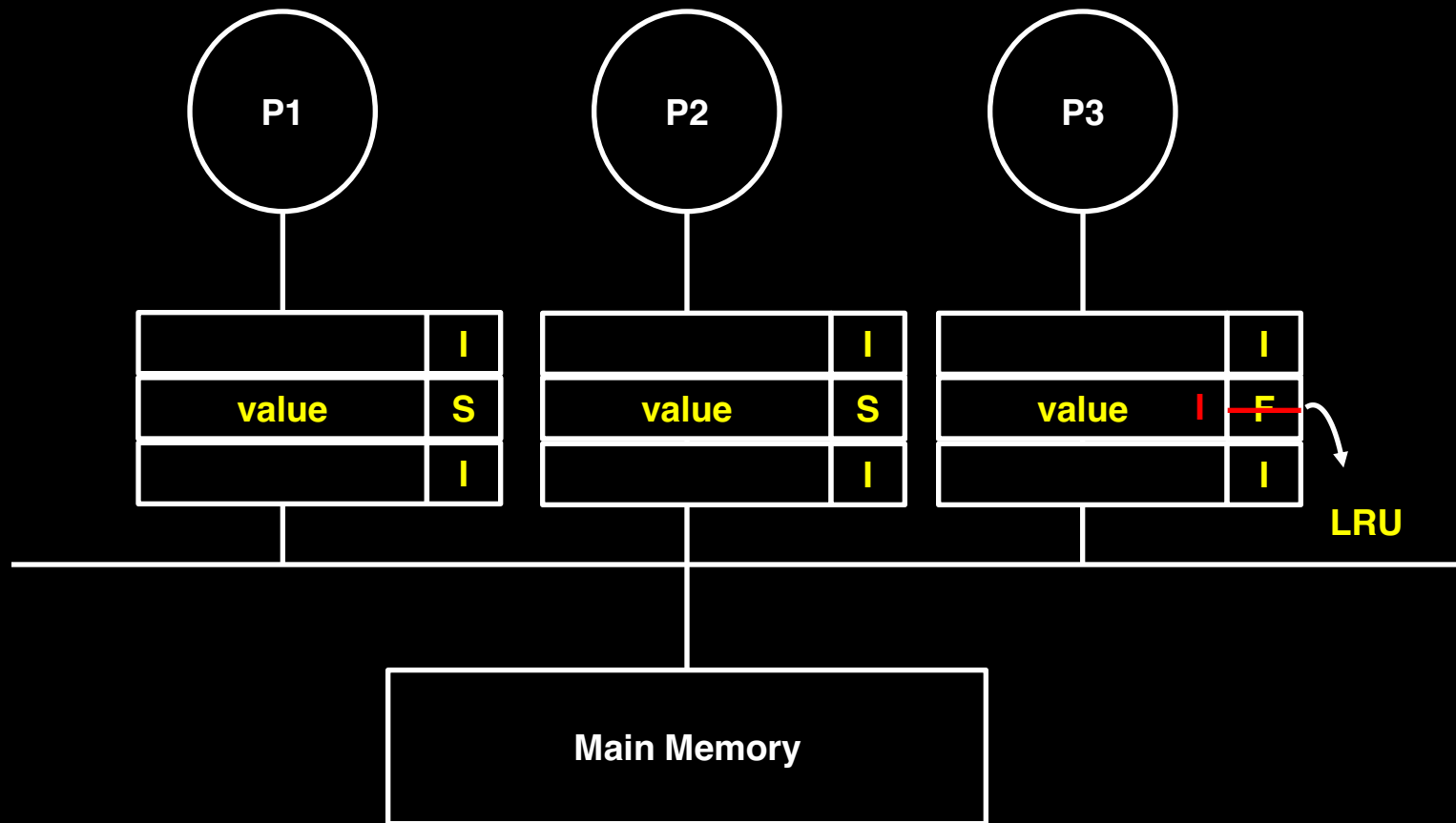
MESIF Protocol Example



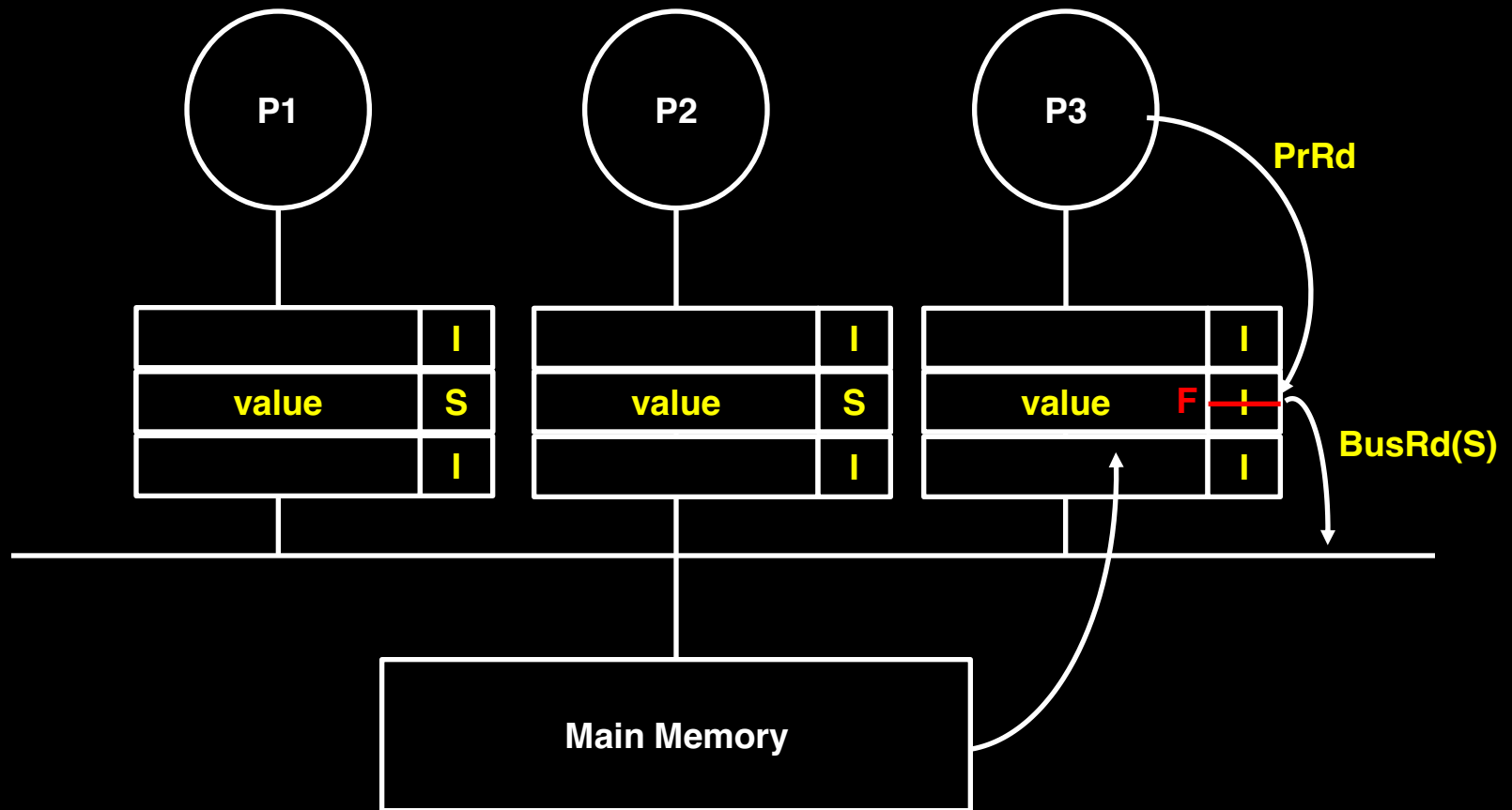
MESIF Protocol Example



MESIF Protocol Example



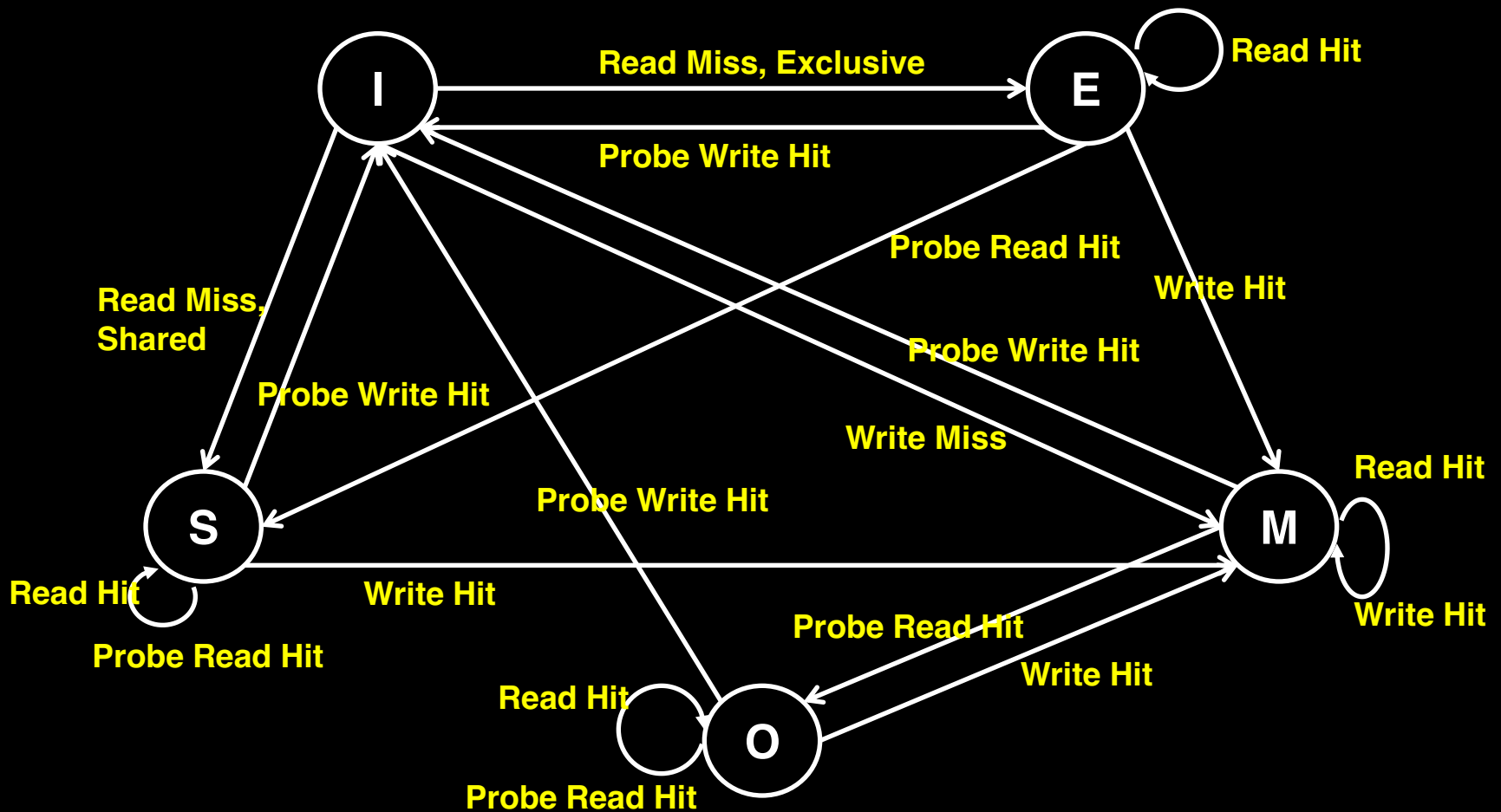
MESIF Protocol Example



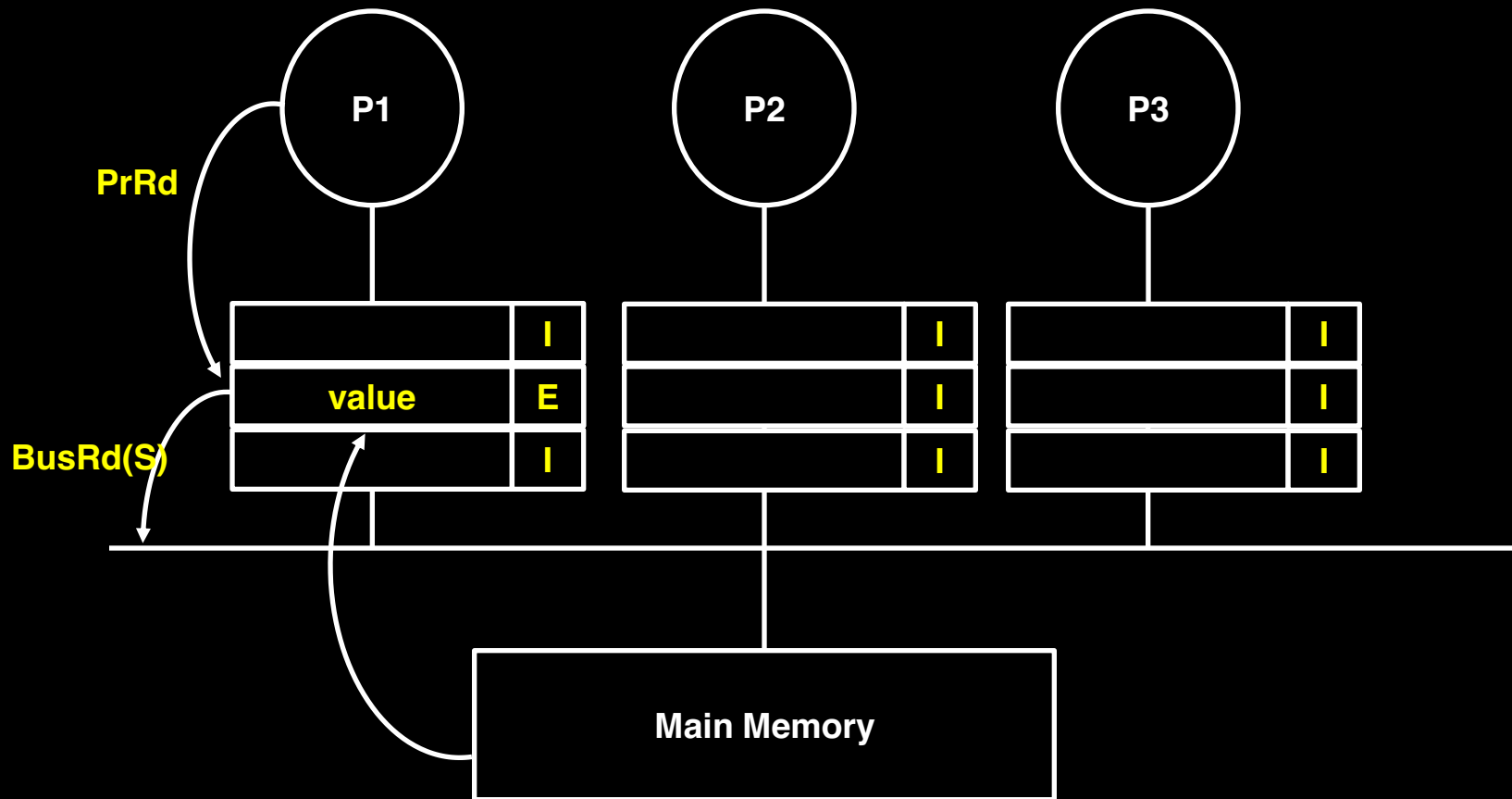
MOESI Protocol

- **Five possible state**
 - Modified
 - Owned
 - Exclusive
 - Shared
 - Invalid
- **The M, E and I states are the same as in the MESI protocol**
- **Only one processor can hold the data in the Owned state—all other processors must hold the data in the Shared state**

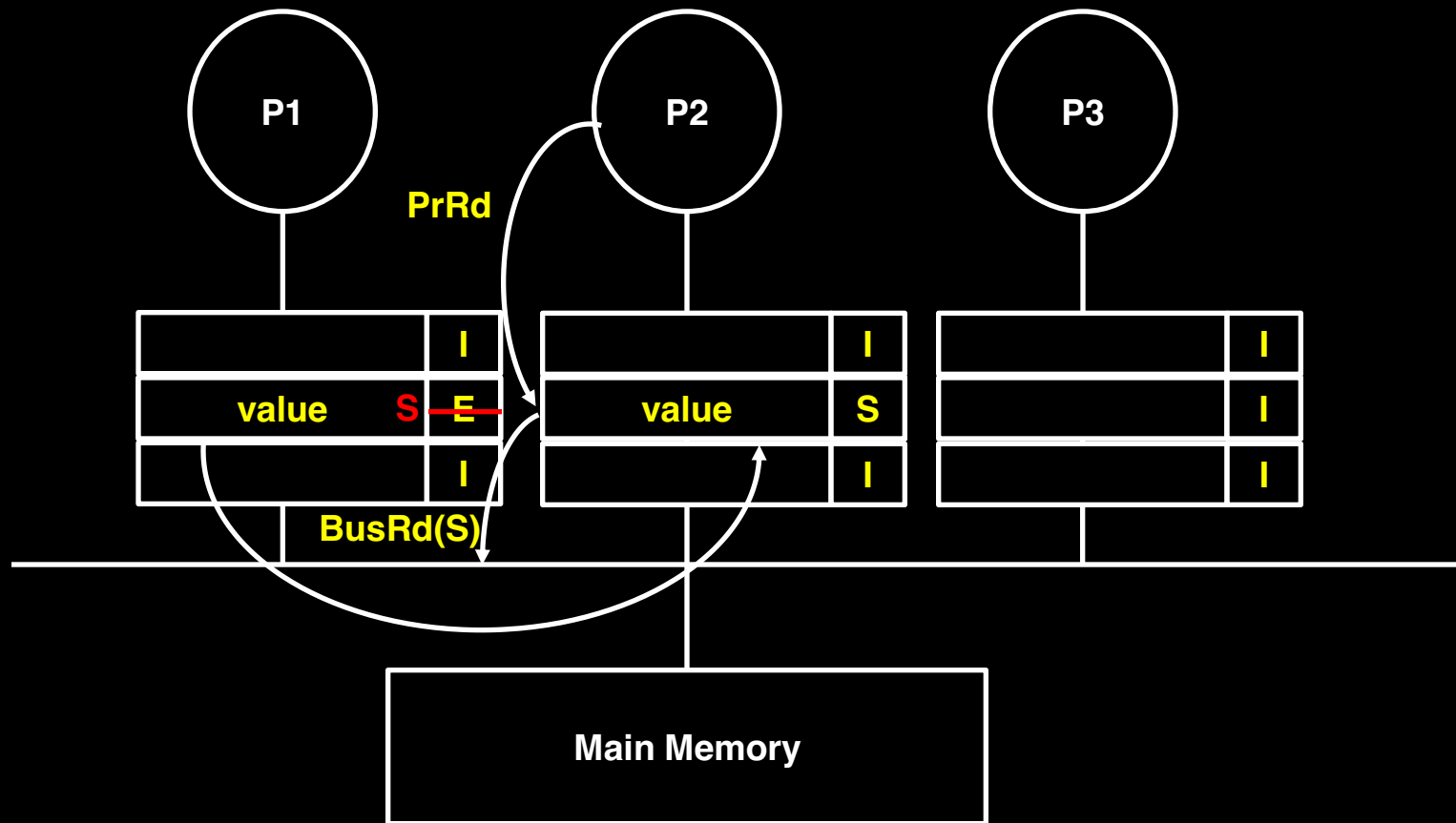
State Diagram



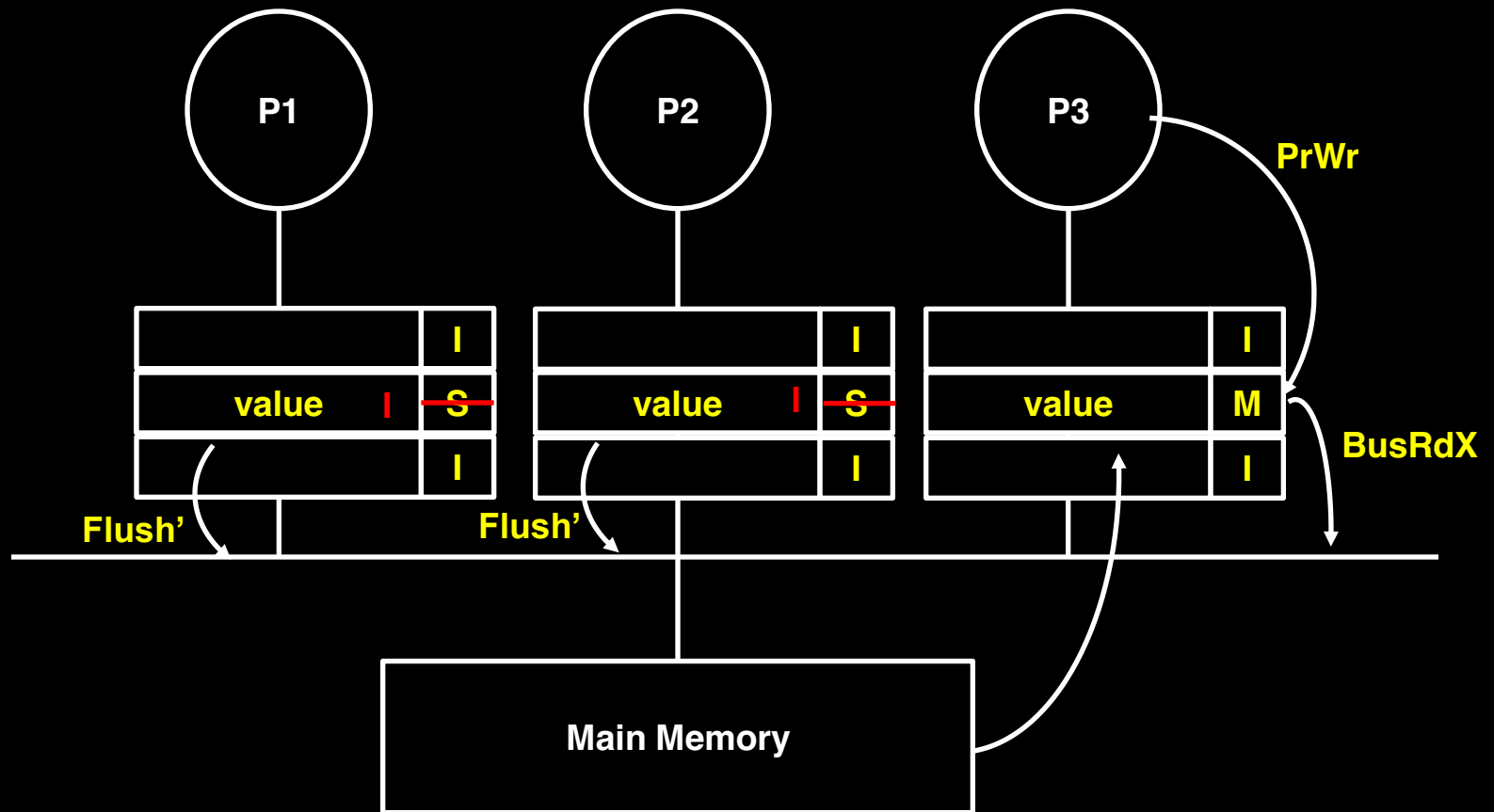
MOESI Protocol Example



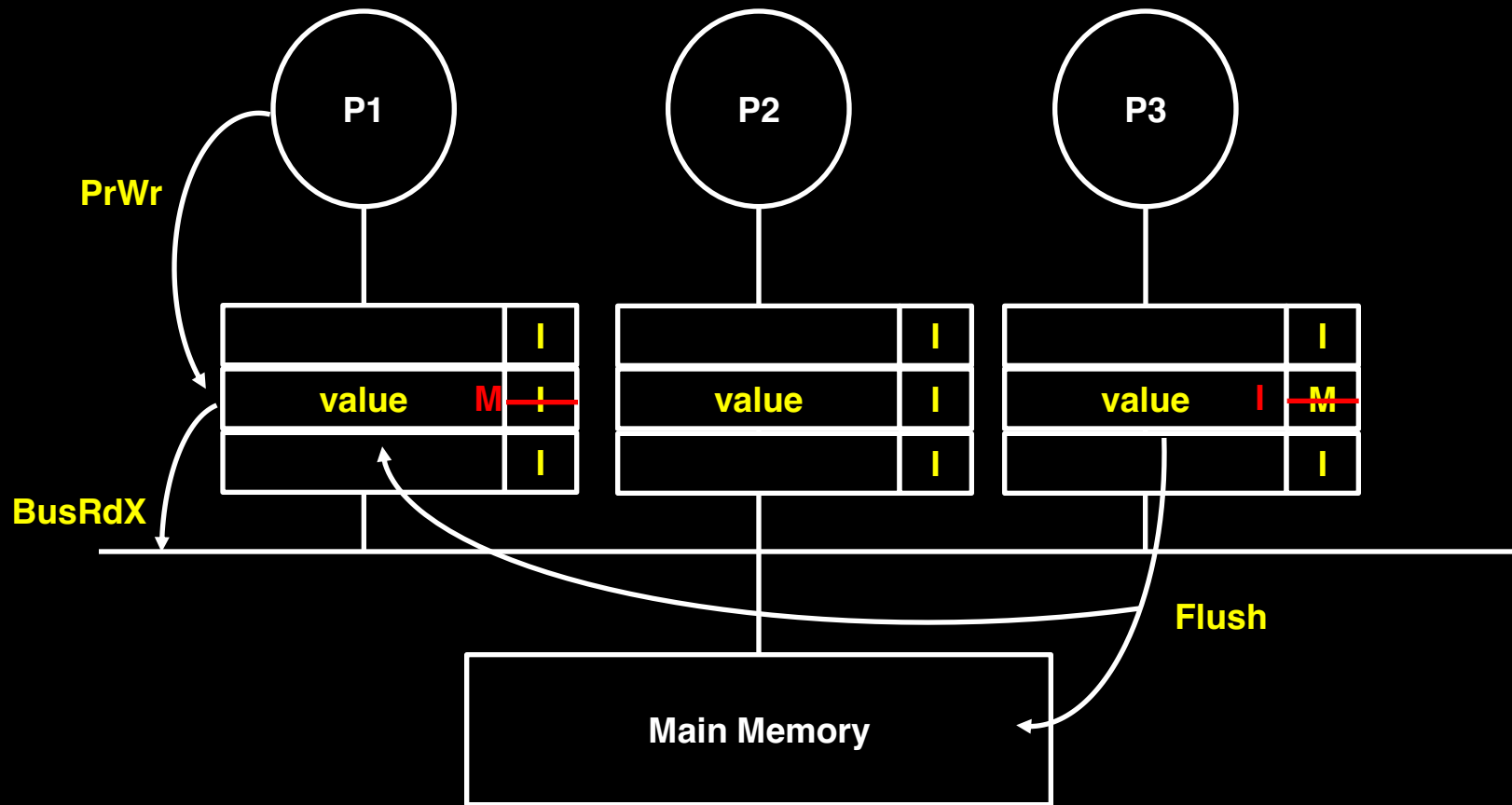
MOESI Protocol Example



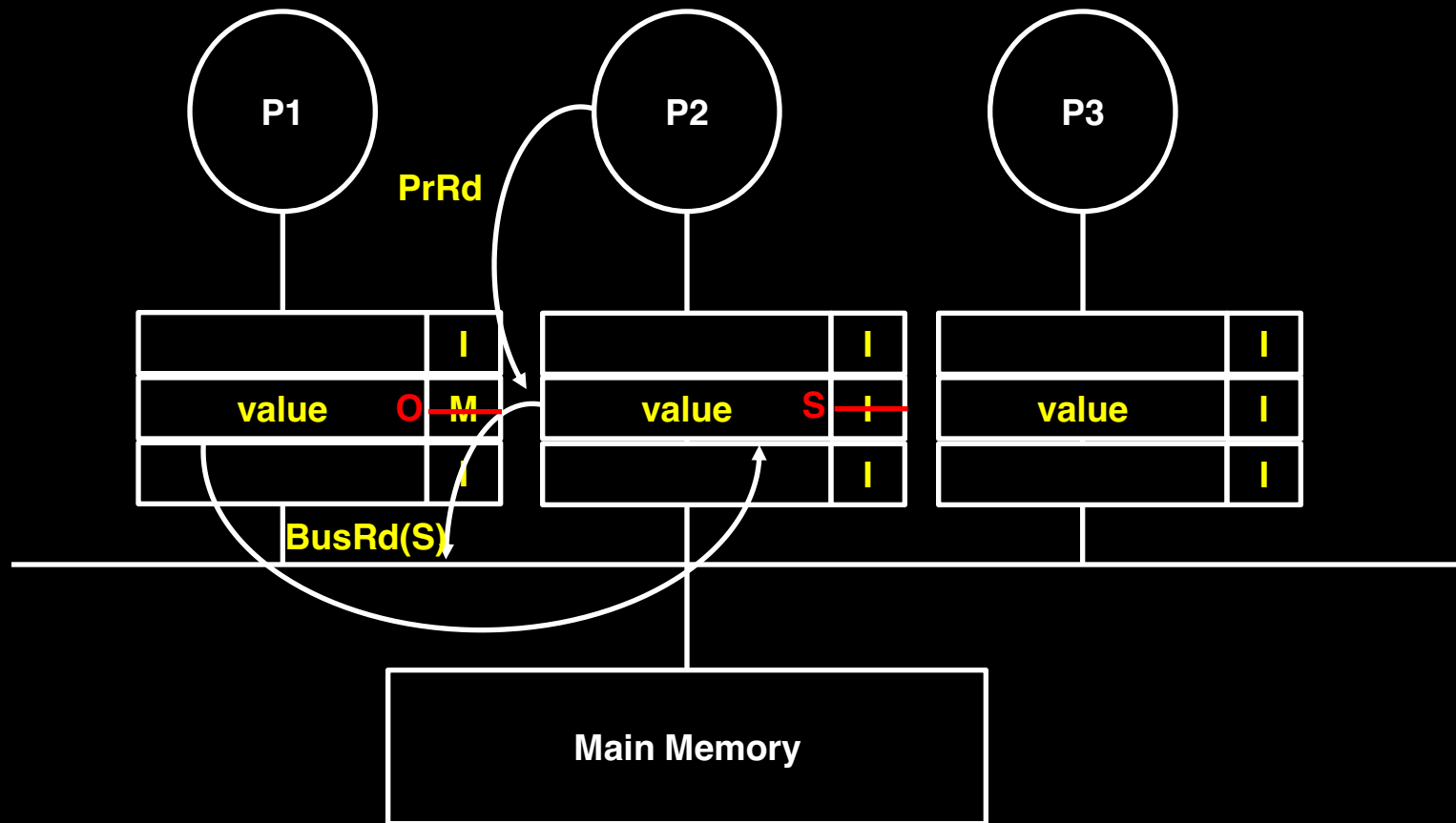
MOESI Protocol Example



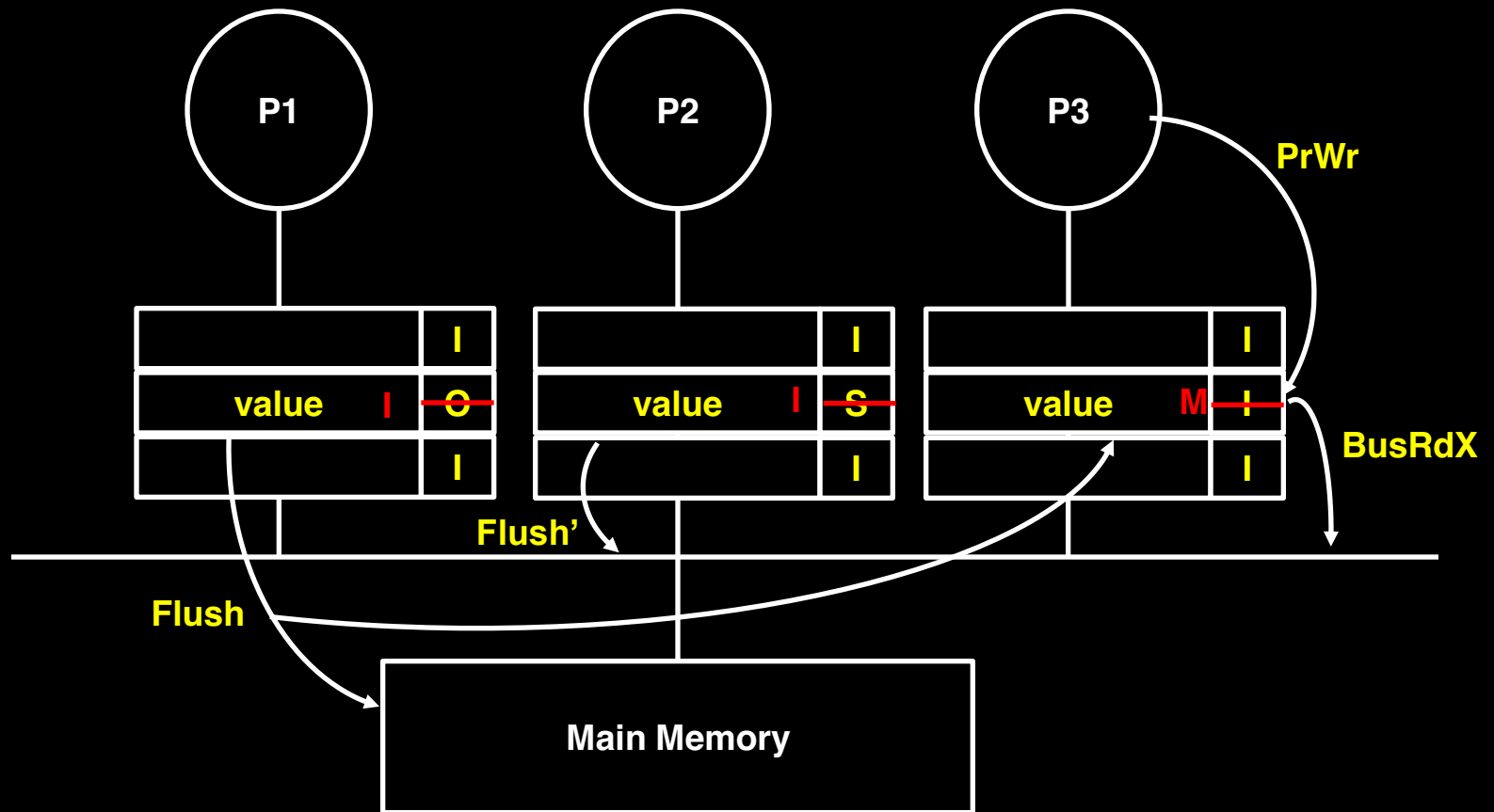
MOESI Protocol Example



MOESI Protocol Example



MOESI Protocol Example



Outline

- Introduction
- Shared Memory Multiprocessor
- Cache Coherence Problem
- **Coherence Summary**

Coherence Summary(1/2)

MSI	Clean/Dirty	Unique?	Can Forward?	Can Silent Transition to
Modified	Dirty	Yes	Yes	
Shared	Clean	No	Yes	I
Invalid	NA	NA	NA	

MESI	Clean/Dirty	Unique?	Can Forward?	Can Silent Transition to
Modified	Dirty	Yes	Yes	
Exclusive	Clean	Yes	Yes	MSI
Shared	Clean	No	Yes	I
Invalid	NA	NA	NA	

Coherence Summary(2/2)

MESIF	Clean/Dirty	Unique?	Can Forward?	Can Silent Transition to
Modified	Dirty	Yes	Yes	
Exclusive	Clean	Yes	Yes	MSI
Shared	Clean	No	No	I
Invalid	NA	NA	NA	
Forward	Clean	Yes	Yes	SI

MOESI	Clean/Dirty	Unique?	Can Forward?	Can Silent Transition to
Modified	Dirty	Yes	Yes	O
Owned	Dirty	Yes	Yes	
Exclusive	Clean	Yes	Yes	MSI
Shared	Either	No	No	I
Invalid	NA	NA	NA	